



SAPIENZA
UNIVERSITÀ DI ROMA

Leveraging IMU for LiDAR odometry through Unscented Kalman Filter

Facoltà di Ingegneria

Laurea Magistrale in Artificial Intelligence and Robotics

Giordano Pagano

Matricola 2077179

Relatore

Dott. Giorgio Grisetti

Anno Accademico 2025/2026

Tesi discussa il 23 Luglio 2026
di fronte a una commissione esaminatrice composta da:
Prof. Mecella (presidente)
Prof. Grisetti
Prof. Amerini
Prof. Anagnostopoulos
Prof. Poggi
Prof. Scardapane
Prof. Silvestri
Prof. Suriani

Leveraging IMU for LiDAR odometry through Unscented Kalman Filter
Tesi sperimentale. Sapienza Università di Roma

© 2026 Giordano Pagano. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: giordypagano@libero.it

*Dedico questo mio grande successo a Dio
che mi ha guidato giorno e notte
e a tutti i miei amici e compagni di viaggio*

Indice

Acronomi	vii
1 Introduction	1
1.1 Estimation Problem Approach	1
1.2 Motivation	2
1.2.1 IMU	2
1.2.2 LiDAR	3
1.2.3 Thesis Objective	4
2 Background	5
2.1 Bayesian Filtering	5
2.1.1 Filtering vs Smoothing vs Prediction	5
2.1.2 Dynamic Systems and Probabilistic Modeling	5
2.1.3 Belief State	6
2.1.4 Filtering Problem	6
2.1.5 Recursive Bayes Filter	6
2.1.6 Markov Assumption	7
2.1.7 Discussion	7
2.2 Kalman Filter	8
2.3 Extended Kalman Filter	8
2.4 Unscented Kalman Filter	9
2.4.1 Unscented Kalman Filter on Manifolds	10
2.4.2 Comparison between EKF and UKF	12
2.5 Iterative Closest Point (ICP)	12
2.5.1 Problem Formulation	12
2.5.2 Algorithm Overview	12
2.5.3 Convergence Properties	13
2.5.4 Sensitivity to Noise and Outliers	13
2.5.5 Variants of ICP	13
2.5.6 Discussion	14
2.6 KD-Tree	14
2.6.1 Definition	14
2.6.2 Construction	14
2.6.3 Nearest Neighbor Search	15
2.6.4 Application in ICP	15
2.6.5 Discussion	15

2.7	LiDAR Odometry	15
2.7.1	Complete Pipeline	16
2.7.2	MAD-ICP Formulation	17
2.7.3	Velocity Estimation	18
3	Leveraging IMU for LiDAR odometry through Unscented Kalman Filter	21
3.1	UKF: State Initialization	21
3.1.1	INIT 1: Pure Inertial Static Initialization	22
3.1.2	INIT 2: IMU Preintegration with GPS Constraints	22
3.1.3	INIT 3: Extended Window Initialization	23
3.1.4	INIT 4: Adaptive Motion Detection	23
3.1.5	INIT 5: Cold Start Initialization	23
3.1.6	INIT 6: Joint Estimation of Gravity and Accelerometer Bias	24
3.1.7	Discussion	24
3.2	Sensor Fusion: LiDAR Inertial Odometry	25
3.3	Sensor Fusion: Implementation details	25
3.3.1	Software Architecture	25
3.3.2	Data Structures	26
3.3.3	Dataset Loading and Synchronization	27
3.3.4	IMU Initialization	28
3.3.5	State Prediction	30
3.3.6	Motion Compensation	31
3.3.7	MAD-ICP Integration	33
3.3.8	UKF Correction	35
3.3.9	Complete Pipeline	37
4	Experiments and Results	39
4.1	Preliminary Experiments: GNSS Inertial Odometry with UKF . . .	39
4.1.1	Synthetic Dataset Generation	40
4.1.2	Unscented Kalman Filter Initialization	42
4.1.3	Unscented Kalman Filter Prediction	42
4.1.4	GNSS Measurement Update	42
4.1.5	Trajectory Evaluation Metrics	43
4.1.6	Output Generation	45
4.1.7	Experimental Setup: NavState VS NavStateExtended	47
4.2	Preliminary Experiments: Results	47
4.2.1	Analysis of Filter Performance under Varying Bias Multipliers	47
4.2.2	Analysis of Filter Robustness under Varying Trajectory Amplitude	50
4.2.3	Initialization Sensitivity Analysis	53
4.3	LiDAR-Inertial Odometry Performance Evaluation	56
4.3.1	IMU Initialization and Local Frame Alignment	56
4.3.2	Methodology and Metrics	57
4.3.3	Experimental Setup and Dataset	57
4.3.4	Quantitative Results	58

Indice	v
5 Conclusion	66
5.1 Final Considerations	66
5.1.1 Biases Analysis	66
5.1.2 Initialization	66
5.1.3 LiDAR-Inertial Odometry	67
Glossario	68
Bibliografia	69
Figure	70
Ringraziamenti	70

Elenco delle figure

4.1	Performance comparison between NavState and NavStateExtended.	48
4.2	bias = 1; NavState (yellow) NavStateExtended (light-blue)	48
4.3	bias = 5; NavState (yellow) and NavStateExtended (light-blue) . . .	49
4.4	bias = 10; NavState (yellow) and NavStateExtended (light-blue) . .	49
4.5	bias = 100; NavState (yellow) and NavStateExtended (light-blue) . .	50
4.6	bias = 100, another point of view	50
4.8	amplitude = 5; NavState (yellow) NavStateExtended (light-blue) . .	51
4.9	amplitude = 10; NavState (yellow) and NavStateExtended (light-blue)	52
4.10	amplitude = 20; NavState (yellow) and NavStateExtended (light-blue)	52
4.11	amplitude = 100; NavState (yellow) and NavStateExtended (light-blue)	52
4.12	INIT 1; GroundTruth (violet) NavStateExtended (green)	53
4.13	INIT 2; GroundTruth (violet) NavStateExtended (green)	54
4.14	INIT 3; GroundTruth (violet) NavStateExtended (green)	54
4.15	INIT 4; GroundTruth (violet) NavStateExtended (green)	54
4.16	INIT 5; GroundTruth (violet) NavStateExtended (green)	55
4.17	INIT 6; GroundTruth (violet) NavStateExtended (green)	55
4.18	EVO analysis of MAD-ICP estimated trajectory	59
4.19	EVO analysis of FAST-LIO estimated trajectory	59
4.20	EVO analysis of UKF-LIO estimated trajectory	60
4.21	EVO analysis of MAD-ICP estimated trajectory	61
4.22	Qualitative evaluation of FAST-LIO estimated trajectory.	61
4.23	EVO analysis of UKF-LIO estimated trajectory	62
4.24	EVO analysis of MAD-ICP estimated trajectory	62
4.25	Qualitative analysis of MAD-ICP estimated point cloud during chal- lenging sequences.	63
4.26	EVO analysis of FAST-LIO estimated trajectory	64
4.27	Qualitative evaluation of UKF-LIO estimation in a challenging scenario.	64

Elenco delle tabelle

3.1	Main software modules of the proposed LiDAR-Inertial pipeline. . .	26
4.1	Trajectory files generated by the framework.	46
4.2	Initialization errors with respect to the ground-truth initial state. . .	55
4.3	QUAD EASY Trajectory Results	58
4.4	QUAD MEDIUM Trajectory Results	60
4.5	QUAD HARD Trajectory Results	62

Capitolo 1

Introduction

The ability to estimate the position and orientation of a moving system is a fundamental requirement in a wide range of modern technological applications. Autonomous vehicles, mobile robots, unmanned aerial vehicles (UAVs), autonomous underwater vehicles, augmented reality systems, and even consumer-grade smartphones continuously rely on localization algorithms to understand their motion within the surrounding environment.

At the core of these technologies lies the problem of odometry, namely the estimation of the trajectory traversed by a system over time. More formally, odometry aims at reconstructing the sequence of poses of a moving platform by exploiting measurements acquired from one or more onboard sensors. The resulting trajectory estimate is then used by higher-level modules responsible for navigation, mapping, planning, and control.

Reliable odometry remains a challenging problem due to the presence of sensor noise, environmental disturbances, model uncertainties, and computational constraints. In practical scenarios, no sensor provides perfect information. Measurements are inherently corrupted by noise and often suffer from biases, drift, limited field of view, or temporary failures. As a consequence, obtaining accurate and robust pose estimates requires sophisticated sensor processing and data fusion techniques.

1.1 Estimation Problem Approach

Accurate trajectory estimation is inherently an uncertain process. Every sensing modality is affected by noise, calibration errors, limited resolution, and environmental disturbances. Consequently, odometry cannot be addressed solely as a geometric reconstruction problem, but must also be viewed as a probabilistic state estimation problem.

State estimation techniques seek to infer the true state of a dynamic system from noisy observations. In the context of navigation and odometry, the state typically includes quantities such as position, velocity, orientation, and sensor biases. Since these quantities cannot be directly observed without uncertainty, the estimation process must explicitly model the probabilistic nature of both sensor measurements and system dynamics.

Among the numerous estimation techniques developed over the past decades, the Kalman Filter has emerged as one of the most influential and widely adopted methods. Its popularity stems from a combination of desirable properties, including mathematical elegance, computational efficiency, robustness, and optimality under linear-Gaussian assumptions. For linear systems, the Kalman Filter provides the minimum mean squared error estimate of the system state by recursively combining predictions from a dynamic model with information provided by sensor measurements.

Unfortunately, most real-world navigation systems exhibit highly nonlinear dynamics. Rotational motion, coordinate transformations, sensor observation models, and geometric registration algorithms all introduce nonlinear relationships that cannot be adequately described by linear models. As a consequence, directly applying the classical Kalman Filter is generally not possible.

The most common solution to this problem has traditionally been the Extended Kalman Filter (EKF), which approximates nonlinear functions through first-order Taylor expansions. Although the EKF has been successfully employed in countless applications, decades of practical experience have highlighted several limitations. The linearization process may introduce significant approximation errors, especially when the underlying dynamics exhibit strong nonlinear behavior. Furthermore, deriving and maintaining analytical Jacobian matrices can become increasingly difficult as system complexity grows.

To overcome these limitations, alternative nonlinear filtering approaches have been proposed. Among them, the Unscented Kalman Filter (UKF) has attracted considerable attention within the robotics, navigation, and control communities. Rather than linearizing nonlinear functions, the UKF approximates the probability distribution itself through a deterministic sampling strategy known as the Unscented Transform. By propagating a carefully selected set of samples through the nonlinear system dynamics, the UKF is capable of capturing higher-order statistical information while avoiding the computation of Jacobian matrices. As a result, it generally provides more accurate state estimates than the EKF when strong nonlinearities are present.

1.2 Motivation

The ability of the UKF to accurately handle nonlinear dynamics makes it particularly attractive for odometry applications, where rotational motion, inertial sensing, and geometric registration naturally introduce nonlinear relationships. These characteristics motivate the investigation of UKF-based estimation frameworks as a foundation for robust trajectory reconstruction systems.

Among the numerous sensing technologies available today, inertial sensors (IMU) and laser sensors (LiDAR) represent two of the most widely adopted solutions for motion estimation. Each sensing modality offers complementary characteristics, strengths, and weaknesses, motivating the development of hybrid approaches capable of exploiting the advantages of both.

1.2.1 IMU

Inertial Measurement Units (IMUs) provide high-frequency measurements of angular velocities and linear accelerations. By integrating these quantities over time, it

is possible to estimate the evolution of the system state. Furthermore, Global Navigation Satellite Systems (GNSS), such as GPS, can provide absolute position measurements in outdoor environments. The fusion of inertial and GNSS information has been extensively studied and constitutes the foundation of many modern navigation systems.

However, inertial navigation systems suffer from a fundamental limitation. Since acceleration and angular velocity measurements must be integrated to recover position and orientation, even small sensor biases accumulate over time, leading to estimation drift. While GNSS measurements can partially compensate for this effect, their availability cannot always be guaranteed. Urban canyons, tunnels, indoor environments, dense vegetation, and adverse weather conditions may significantly degrade GNSS performance or completely prevent signal reception.

1.2.2 LiDAR

To address these limitations, increasing attention has been devoted to perception-based odometry methods. Among them, LiDAR odometry has emerged as one of the most successful approaches for accurate motion estimation in complex environments. LiDAR sensors directly measure the geometry of the surrounding scene by emitting laser pulses and observing their reflections. The resulting point clouds provide rich geometric information that can be exploited to estimate the relative motion between consecutive observations.

Over the last two decades, the research community has proposed numerous LiDAR odometry algorithms capable of achieving remarkable accuracy in both indoor and outdoor environments. Most existing solutions rely on point cloud registration techniques, where consecutive scans are aligned to estimate the sensor displacement.

Recently, more advanced systems have been proposed to improve robustness under challenging operating conditions. A notable example is MAD-ICP, a LiDAR odometry framework specifically designed to operate reliably across different environments, sensor configurations, and motion patterns. Unlike many traditional approaches that implicitly rely on assumptions regarding the operating conditions, MAD-ICP exploits a PCA-based KD-tree representation to extract structural information from the observed point clouds and compute robust alignment metrics. Furthermore, uncertainty-aware local map management is employed to limit trajectory drift while maintaining real-time performance.

At the same time, the robotics community has witnessed the emergence of tightly-coupled LiDAR-Inertial Odometry systems, which combine geometric information from LiDAR sensors with the high-frequency motion information provided by IMUs. One of the most successful examples is FAST-LIO, a framework capable of achieving highly accurate pose estimation while maintaining computational efficiency suitable for real-time applications. FAST-LIO directly registers raw LiDAR points to a continuously updated map, avoiding handcrafted feature extraction and improving adaptability across different LiDAR technologies. Moreover, its incremental KD-tree data structure enables efficient map maintenance and fast nearest-neighbor queries, allowing the system to operate at frequencies exceeding 100 Hz while maintaining robustness under aggressive rotational motions.

Despite the remarkable progress achieved by modern LiDAR odometry and LiDAR-Inertial Odometry systems, several challenges remain open. In particular, aggressive motions characterized by rapid changes in velocity and angular rate may still negatively affect estimation accuracy. Large rotational motions can temporarily degrade scan registration quality, while highly dynamic trajectories may challenge the assumptions underlying many registration and filtering algorithms.

These observations motivate the development of estimation frameworks capable of leveraging the complementary strengths of geometric registration techniques and probabilistic state estimation methods. While LiDAR-based registration provides accurate relative pose constraints derived from environmental geometry, probabilistic filters offer a principled framework for modeling uncertainty, incorporating prior knowledge, and exploiting dynamic system models.

1.2.3 Thesis Objective

The central hypothesis of this thesis is that combining the geometric robustness of LiDAR scan registration with the probabilistic consistency of Unscented Kalman filtering can yield a trajectory estimation framework capable of maintaining accuracy under aggressive motions and highly nonlinear operating conditions.

To investigate this hypothesis, the proposed approach is developed through a progressive analysis of increasingly sophisticated odometry systems. First, a GNSS-Inertial Odometry framework based on the Unscented Kalman Filter is studied to analyze the mechanisms governing nonlinear probabilistic state estimation and the contribution of inertial measurements to trajectory reconstruction. Subsequently, the MAD-ICP algorithm is investigated as a standalone LiDAR odometry solution, focusing on its registration strategy and uncertainty management mechanisms. Finally, the two approaches are combined into a unified estimation framework aimed at improving robustness and accuracy under demanding operating conditions characterized by strong accelerations and rapid rotational dynamics.

The remainder of this thesis is organized as follows. Chapter 2 introduces the theoretical background required to understand the proposed methodology, including Bayesian filtering, Kalman filtering techniques, ICP-based registration methods, and KD-tree data structures. Chapter 3 presents the investigated odometry frameworks and describes the proposed fusion strategy. Chapter 4 details the experimental setup and evaluation methodology. Chapter 5 discusses the obtained results and analyzes the strengths and limitations of the proposed approach. Finally, Chapter 6 summarizes the main contributions of the thesis and outlines possible directions for future work.

Capitolo 2

Background

2.1 Bayesian Filtering

State estimation in dynamic systems can be formulated within a probabilistic framework⁽⁵⁾. The goal is to estimate the state $\mathbf{X}_t$ of a system at time t , given a sequence of observations and control inputs.

2.1.1 Filtering vs Smoothing vs Prediction

In the context of probabilistic inference for dynamic systems, different estimation problems can be defined depending on the available information.

- **Filtering:** estimation of the current state given past and current observations

$$p(\mathbf{X}_t \mid \mathbf{z}_{1:t}) \quad (2.1)$$

- **Prediction:** estimation of a future state

$$p(\mathbf{X}_{t+k} \mid \mathbf{z}_{1:t}) \quad (2.2)$$

- **Smoothing:** estimation of a past state using all available observations

$$p(\mathbf{X}_t \mid \mathbf{z}_{1:T}), \quad T > t \quad (2.3)$$

Filtering is the central topic chosen for this thesis, with the aim of analyzing a new algorithmic technique capable of exploiting and combining simultaneously the information coming from IMU and LIDAR measurements.

2.1.2 Dynamic Systems and Probabilistic Modeling

A dynamic system evolves over time according to two main components:

- **Transition model:**

$$p(\mathbf{X}_t \mid \mathbf{X}_{t-1}, \mathbf{u}_t) \quad (2.4)$$

which describes how the state evolves given the previous state and control input.

- **Observation model:**

$$p(\mathbf{z}_t \mid \mathbf{X}_t) \quad (2.5)$$

which describes the likelihood of observing $\mathbf{z}_t$ given the current state.

These models define a probabilistic representation of the system, often referred to as a *Dynamic Bayesian Network*, characterized by a recursive structure over time.

2.1.3 Belief State

Since the true state of the system is not directly observable, filtering algorithms maintain a probability distribution over the state, known as the *belief*:

$$bel(\mathbf{X}_t) = p(\mathbf{X}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \quad (2.6)$$

The belief represents all the information available about the system at time t , given past observations and controls.

Depending on the nature of the state space, the belief can be represented in different ways:

- Discrete states: probability mass function (array of probabilities)
- Continuous Gaussian states: mean and covariance
- Arbitrary distributions: sample-based representations (e.g., particles)

The belief can be interpreted as a complete probabilistic description of the system state. Rather than providing a single estimate, it encodes uncertainty explicitly.

For instance, a narrow distribution indicates high confidence in the estimate, while a wide distribution reflects high uncertainty.

This probabilistic representation is crucial in real-world applications, where noise, sensor inaccuracies, and model imperfections are unavoidable.

2.1.4 Filtering Problem

The filtering problem consists in computing the current belief given all past measurements and controls:

$$bel(\mathbf{X}_t) = p(\mathbf{X}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \quad (2.7)$$

This is one of the main inference tasks in dynamic systems, together with smoothing and maximum a posteriori estimation.

2.1.5 Recursive Bayes Filter

The key idea behind Bayesian filtering is that the belief can be updated recursively in two steps:

Prediction Step

The prediction step incorporates the effect of the control input:

$$\overline{bel}(\mathbf{X}_t) = \int p(\mathbf{X}_t | \mathbf{X}_{t-1}, \mathbf{u}_t) bel(\mathbf{X}_{t-1}) d\mathbf{X}_{t-1} \quad (2.8)$$

This step propagates the previous belief through the system dynamics, then the integral marginalizes over all possible previous states, effectively combining all possible state transitions weighted by their probability.

Update Step

The update step incorporates the new observation:

$$bel(\mathbf{X}_t) = \eta p(\mathbf{z}_t | \mathbf{X}_t) \overline{bel}(\mathbf{X}_t) \quad (2.9)$$

where η is a normalization constant ensuring that the belief integrates to one.

2.1.6 Markov Assumption

A key assumption in Bayesian filtering is the *Markov property*, which states that the current state $\mathbf{X}_t$ depends only on the previous state $\mathbf{X}_{t-1}$ and not on earlier states:

$$p(\mathbf{X}_t | \mathbf{X}_{0:t-1}) = p(\mathbf{X}_t | \mathbf{X}_{t-1}) \quad (2.10)$$

Similarly, the observation depends only on the current state:

$$p(\mathbf{z}_t | \mathbf{X}_{0:t}) = p(\mathbf{z}_t | \mathbf{X}_t) \quad (2.11)$$

This assumption makes recursive estimation tractable.

2.1.7 Discussion

Bayesian filtering provides a general framework for state estimation in dynamic systems. However, the recursive equations are often computationally intractable, especially in nonlinear or high-dimensional systems.

This motivates the use of approximate filters, such as:

- Kalman Filter (linear Gaussian case)
- Extended Kalman Filter (first-order approximation)
- Unscented Kalman Filter (higher-order approximation)

In the following sections, these methods are introduced as practical implementations of the Bayesian filtering framework.

2.2 Kalman Filter

The Kalman Filter (KF) provides an optimal solution for linear systems with Gaussian noise.

The system is defined as:

$$\mathbf{X}_t = A\mathbf{X}_{t-1} + B\mathbf{u}_t + \mathbf{w}_t \quad (2.12)$$

$$\mathbf{z}_t = H\mathbf{X}_t + \mathbf{v}_t \quad (2.13)$$

where $\mathbf{w}_t \sim \mathcal{N}(0, Q)$ and $\mathbf{v}_t \sim \mathcal{N}(0, R)$.

Under these assumptions, the belief remains Gaussian:

$$\mathbf{X}_t \sim \mathcal{N}(\hat{\mathbf{X}}_t, \Sigma_t) \quad (2.14)$$

Prediction

$$\hat{\mathbf{X}}_t^- = A\hat{\mathbf{X}}_{t-1} + B\mathbf{u}_t \quad (2.15)$$

$$\Sigma_t^- = A\Sigma_{t-1}A^T + Q \quad (2.16)$$

Update

$$K_t = \Sigma_t^- H^T (H\Sigma_t^- H^T + R)^{-1} \quad (2.17)$$

$$\hat{\mathbf{X}}_t = \hat{\mathbf{X}}_t^- + K_t(\mathbf{z}_t - H\hat{\mathbf{X}}_t^-) \quad (2.18)$$

$$\Sigma_t = (I - K_t H)\Sigma_t^- \quad (2.19)$$

The Kalman gain K_t determines how much the estimate is corrected based on the new measurement.

- If the measurement noise R is small, the gain is high, and the filter relies more on the observation.
- If the process uncertainty is small, the gain is low, and the prediction is trusted more.

Thus, the Kalman filter can be interpreted as an optimal trade-off between prediction and observation.

Despite its optimality, the KF is limited to linear systems. It is a Bayesian Filter that operates in a closed Gaussian world by estimating the parameters of the state distribution as new measurements and controls become available.

2.3 Extended Kalman Filter

In real-world applications, system dynamics are often nonlinear. The Extended Kalman Filter (EKF) extends the KF by linearizing the nonlinear functions through first-order Taylor expansion.

The system becomes:

$$\mathbf{X}_t = f(\mathbf{X}_{t-1}, \mathbf{u}_t) + \mathbf{w}_t \quad (2.20)$$

$$\mathbf{z}_t = h(\mathbf{X}_t) + \mathbf{v}_t \quad (2.21)$$

The Jacobians are computed as:

$$F_t = \left. \frac{\partial f}{\partial \mathbf{X}} \right|_{\hat{\mathbf{x}}_{t-1}} \quad (2.22)$$

$$H_t = \left. \frac{\partial h}{\partial \mathbf{X}} \right|_{\hat{\mathbf{x}}_t^-} \quad (2.23)$$

The EKF then applies the standard Kalman equations using these linear approximations.

Although widely used, the EKF has several limitations:

- It introduces linearization errors
- It requires computation of Jacobians
- It provides only first-order accuracy

These limitations become critical in highly nonlinear systems.

2.4 Unscented Kalman Filter

The Unscented Kalman Filter (UKF)⁽³⁾ addresses the limitations of the EKF by avoiding explicit linearization.

Instead of approximating nonlinear functions, the UKF approximates the probability distribution. This is achieved through the *Unscented Transform*, which represents a Gaussian distribution using a set of deterministically chosen samples called *sigma points*.

Given a Gaussian distribution $\mathcal{N}(\hat{\mathbf{X}}, \Sigma)$, sigma points are generated as:

$$\boldsymbol{\xi}_i = \left(\sqrt{(n + \lambda)\Sigma} \right)_i \quad (2.24)$$

$$\boldsymbol{\xi}_{i+n} = - \left(\sqrt{(n + \lambda)\Sigma} \right)_i \quad (2.25)$$

These points are propagated through the nonlinear system, and the transformed mean and covariance are reconstructed from the propagated samples.

This approach provides a higher-order approximation compared to the EKF and does not require Jacobians, making it a derivative-free method.

Each sigma point is associated with a weight used to reconstruct the mean and covariance.

$$w_0^{(m)} = \frac{\lambda}{n + \lambda} \quad (2.26)$$

$$w_0^{(c)} = \frac{\lambda}{n + \lambda} + (1 - \alpha^2 + \beta) \quad (2.27)$$

$$w_i = \frac{1}{2(n + \lambda)} \quad (2.28)$$

The parameters α and β control the spread of the sigma points:

- α : determines the spread of the sigma points
- β : incorporates prior knowledge of the distribution (optimal value is 2 for Gaussian distributions)

2.4.1 Unscented Kalman Filter on Manifolds

In many applications, the system state lies on a nonlinear manifold rather than in a Euclidean space. A common example is 3D orientation, which belongs to the Lie group $SO(3)$.

To handle such cases, the state must be represented using manifold-aware operations.

State Representation

A manifold $\mathcal{S}$ is locally equivalent to $\mathbb{R}^n$, allowing the definition of local coordinates.

Two operators are introduced:

$$\boxplus : \mathcal{S} \times \mathbb{R}^n \rightarrow \mathcal{S} \quad (2.29)$$

$$\boxminus : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}^n \quad (2.30)$$

They allow:

- applying perturbations: $\mathbf{x} \boxplus \boldsymbol{\delta}$
- computing differences: $\mathbf{y} \boxminus \mathbf{x}$

Uncertainty Representation

Uncertainty is defined in the tangent space:

$$\mathbf{X} = \hat{\mathbf{X}} \boxplus \boldsymbol{\xi}, \quad \boldsymbol{\xi} \sim \mathcal{N}(0, \Sigma) \quad (2.31)$$

subsectionsubsectionPrediction Step

Given:

$$\mathbf{X}_t = f(\mathbf{X}_{t-1}, \mathbf{u}_t, \boldsymbol{\epsilon}_t) \quad (2.32)$$

the mean is propagated as:

$$\hat{\mathbf{X}}_t = f(\hat{\mathbf{X}}_{t-1}, \mathbf{u}_t, 0) \quad (2.33)$$

Sigma points are propagated:

$$\mathbf{X}_t^i = f(\hat{\mathbf{X}}_{t-1} \boxplus \boldsymbol{\xi}_i, \mathbf{u}_t, 0) \quad (2.34)$$

The covariance is computed using:

$$\Sigma_t = \sum_i w_i (\hat{\mathbf{X}}_t \boxminus \mathbf{X}_t^i) (\hat{\mathbf{X}}_t \boxminus \mathbf{X}_t^i)^T \quad (2.35)$$

Update Step

Given a measurement:

$$\mathbf{z} = h(\mathbf{X}) + \boldsymbol{\delta} \quad (2.36)$$

sigma points are projected:

$$\mathbf{z}_i = h(\mathbf{X}_i) \quad (2.37)$$

The Kalman update is then computed using the transformed sigma points, resulting in:

$$\hat{\mathbf{X}}^+ = \hat{\mathbf{X}} \boxplus K(\mathbf{z} - \bar{\mathbf{z}}) \quad (2.38)$$

$$\Sigma^+ = \Sigma - K\Sigma_{zz}K^T \quad (2.39)$$

Discussion

The UKF on manifolds combines the advantages of the Unscented Transform with a geometrically consistent state representation.

Its main advantages are:

- No need for Jacobians
- Better handling of nonlinearities in transition and observation functions
- Natural support for manifold-valued states

2.4.2 Comparison between EKF and UKF

The Extended Kalman Filter and the Unscented Kalman Filter represent two different approaches to handling nonlinear systems.

- EKF: first-order approximation, requires Jacobians
- UKF: higher-order approximation, derivative-free

While the UKF is generally more accurate, it is computationally more expensive. The choice between the two methods depends on the specific application and computational constraints.

2.5 Iterative Closest Point (ICP)

The Iterative Closest Point (ICP)⁽¹⁾ is a widely used method for aligning two sets of 3D points. It is commonly employed in applications such as point cloud registration, pose estimation, and 3D reconstruction.

Given two point sets:

- a *source* point cloud $\mathcal{P} = \{\mathbf{p}_i\}$
- a *target* point cloud $\mathcal{Q} = \{\mathbf{q}_i\}$

the goal of ICP is to find the rigid transformation (rotation $\mathbf{R}$ and translation $\mathbf{t}$) that best aligns $\mathcal{P}$ to $\mathcal{Q}$.

2.5.1 Problem Formulation

The alignment problem can be formulated as the minimization of the following cost function:

$$E(\mathbf{R}, \mathbf{t}) = \sum_i \|\mathbf{q}_i - (\mathbf{R}\mathbf{p}_i + \mathbf{t})\|^2 \quad (2.40)$$

where $(\mathbf{R}, \mathbf{t})$ define a rigid transformation in 3D space.

2.5.2 Algorithm Overview

ICP is an iterative algorithm that alternates between two main steps:

Correspondence Estimation

For each point $\mathbf{p}_i \in \mathcal{P}$, the closest point $\mathbf{q}_i \in \mathcal{Q}$ is found:

$$\mathbf{q}_i = \arg \min_{\mathbf{q} \in \mathcal{Q}} \|\mathbf{q} - \mathbf{p}_i\| \quad (2.41)$$

This step establishes correspondences between the two point sets.

Transformation Estimation

Given the correspondences, the optimal rigid transformation is computed by minimizing the cost function. This can be solved in closed form using methods such as Singular Value Decomposition (SVD).

Iteration

The source point cloud is transformed using the estimated transformation, and the process is repeated until convergence.

2.5.3 Convergence Properties

ICP typically converges to a local minimum of the error function. Its performance depends on:

- the initial alignment between the point clouds
- the quality of correspondences
- the presence of noise and outliers

A good initial guess is often required to ensure convergence to the correct solution.

2.5.4 Sensitivity to Noise and Outliers

In practical scenarios, point clouds are often affected by noise and outliers.

Incorrect correspondences can significantly degrade the performance of ICP. To mitigate this issue, several strategies can be adopted:

- rejection of correspondences based on distance thresholds
- use of robust cost functions
- weighting schemes for correspondences

2.5.5 Variants of ICP

Several variants of ICP have been proposed to improve robustness and accuracy:

- **Point-to-Point ICP:** minimizes Euclidean distances between corresponding points
- **Point-to-Plane ICP:** minimizes distances along surface normals, improving convergence speed
- **Generalized ICP:** models local surface structure using covariance matrices

2.5.6 Discussion

ICP provides a practical solution for point cloud alignment, but it has some limitations:

- sensitivity to initialization
- susceptibility to local minima
- dependence on correct correspondences

Despite these limitations, ICP remains a fundamental tool in 3D perception pipelines and is often combined with probabilistic methods, such as Kalman filters, to improve robustness in dynamic scenarios.

In this work, ICP is used to estimate the relative transformation between 3D observations, providing measurements that are later integrated within a probabilistic filtering framework.

2.6 KD-Tree

Efficient nearest neighbor search is a fundamental component in many geometric algorithms, including the Iterative Closest Point (ICP). A common data structure used to accelerate these queries is the *k-dimensional tree* (KD-Tree)⁽²⁾.

2.6.1 Definition

A KD-Tree is a binary space-partitioning data structure used to organize points in a k -dimensional space. It recursively partitions the space into hyperplanes, allowing efficient search operations.

Each node of the tree represents a point and splits the space along one of the coordinate axes. The splitting dimension typically alternates at each level of the tree.

2.6.2 Construction

Given a set of points, the KD-Tree is constructed recursively:

- Select a splitting dimension (e.g., cycling through axes)
- Choose a pivot point (commonly the median)
- Partition the dataset into two subsets:
 - points with smaller coordinate values
 - points with larger coordinate values

This process produces a balanced tree when median splitting is used, resulting in a depth of approximately $\mathcal{O}(\log N)$.

2.6.3 Nearest Neighbor Search

The KD-Tree enables efficient nearest neighbor queries. Given a query point $\mathbf{p}$, the search proceeds as follows:

- Traverse the tree to reach the leaf node corresponding to $\mathbf{p}$
- Keep track of the current best candidate
- Backtrack through the tree, checking whether other branches could contain closer points

This pruning strategy significantly reduces the number of distance computations. The average time complexity for nearest neighbor search is:

$$\mathcal{O}(\log N) \quad (2.42)$$

although in the worst case it can degrade to $\mathcal{O}(N)$.

2.6.4 Application in ICP

In the ICP algorithm, nearest neighbor search is required at each iteration to establish correspondences between point clouds.

Using a KD-Tree to store the target point cloud $\mathcal{Q}$ allows efficient computation of:

$$\mathbf{q}_i = \arg \min_{\mathbf{q} \in \mathcal{Q}} \|\mathbf{q} - \mathbf{p}_i\| \quad (2.43)$$

This reduces the computational cost from $\mathcal{O}(N^2)$ to approximately $\mathcal{O}(N \log N)$ per iteration.

2.6.5 Discussion

KD-Trees provide an effective solution for accelerating nearest neighbor queries in low- to medium-dimensional spaces, such as 3D point clouds.

However, their performance decreases in higher dimensions due to the *curse of dimensionality*. In such cases, alternative methods such as approximate nearest neighbor search may be preferred.

In this work, KD-Trees are used to efficiently compute point correspondences within the ICP framework, enabling real-time performance in 3D registration tasks.

2.7 LiDAR Odometry

The LiDAR registration component adopted in this work is based on the *MAD-ICP*⁽⁴⁾ (Matching Data - Iterative Closest Point) framework. MAD-ICP is a registration algorithm specifically designed for real-time LiDAR odometry and mapping, combining efficient spatial data structures with probabilistic point cloud matching.

Unlike classical ICP variants, which establish point-to-point or point-to-plane correspondences directly between consecutive scans, MAD-ICP represents the environment through a hierarchical collection of local geometric distributions. This representation enables robust registration while maintaining computational efficiency, making the algorithm suitable for real-time robotic applications.

2.7.1 Complete Pipeline

The proposed lidar-only odometry pipeline is built around a robust ICP formulation operating on a hierarchical spatial representation of the incoming LiDAR scans. The system incrementally estimates the sensor pose with respect to a global map while maintaining a short-term memory of keyframes and an adaptive velocity model.

System Overview

At each time step, the pipeline receives a raw LiDAR point cloud and the corresponding timestamp. The system maintains the current estimate of the sensor pose $T_{k \rightarrow map}$, a set of keyframes, and a trajectory history. If the system is not yet initialized, the first incoming scan is converted into a map representation and used to bootstrap the state.

Motion Compensation (Deskewing)

To compensate for motion distortion within a single LiDAR scan, a deskewing procedure is applied when enabled. Given an estimate of the sensor velocity, each point is corrected by interpolating the rigid body motion over the scan duration:

$$\mathbf{T}(t) = \exp(\xi t)$$

where ξ is a 6D velocity twist estimated from the previous motion.

Spatial Representation

Each point cloud is converted into a hierarchical structure (MAD-tree), which partitions the space into adaptive regions. The leaves of this structure represent the minimal units used for registration.

Initial Pose Prediction

Before optimization, an initial guess is computed using a constant velocity model:

$$\mathbf{T}_k^0 = \mathbf{T}_{k-1} \cdot \exp(\hat{\mathbf{v}} \Delta t)$$

where $\hat{\mathbf{v}}$ is the estimated velocity and Δt is the sensor sampling period.

MAD-ICP Optimization

The core registration is performed using a robust ICP formulation. At each iteration, correspondences between the current scan and the map are established through the hierarchical structure, and the pose is updated by minimizing a robust cost function:

$$\mathbf{T}^* = \arg \min_{\mathbf{T}} \sum_i \rho(\|\mathbf{p}_i - \mathbf{T}\mathbf{q}_i\|^2)$$

where $\rho(\cdot)$ is a robust weighting function derived from MAD statistics.

The optimization is iterated for a fixed number of steps or until real-time constraints are reached.

Velocity Estimation

After convergence, the resulting trajectory is used to update a local velocity estimate through a smoothing window of recent poses. This velocity is then reused in the next iteration for deskewing and prediction.

Keyframe Selection and Map Update

A new keyframe is inserted when the ratio of inlier correspondences falls below a threshold. Keyframes are used to maintain a compact representation of the environment while ensuring robustness to degeneracies.

Pipeline Output

The system outputs:

- The current pose estimate $\mathbf{T}_{k \rightarrow map}$
- The trajectory history
- The set of active keyframes
- The estimated motion velocity

2.7.2 MAD-ICP Formulation

The registration step is performed using a robust point-to-plane ICP formulation operating on a hierarchical spatial representation of the environment. Each LiDAR scan is encoded as a set of leaf nodes extracted from a MAD-tree structure, where each node contains a representative mean point and an associated surface normal estimated from local eigen analysis.

Geometric Error Model

Given a transformed point $\mathbf{p}_i$ from the moving scan and its corresponding matched point $\mathbf{q}_i$ in the fixed map, the residual is defined in a point-to-plane manner:

$$\mathbf{e}_i = (\mathbf{T}\mathbf{p}_i - \mathbf{q}_i) \cdot \mathbf{n}_i$$

where $\mathbf{n}_i$ is the normal vector associated with the fixed correspondence.

Jacobian Derivation

The optimization is performed in the Lie algebra $\mathfrak{se}(3)$, where the transformation is parametrized as $\mathbf{T} = \exp(\xi)$. The Jacobian of the residual with respect to the pose increment ξ is given by:

$$\mathbf{J}_i = \begin{bmatrix} \mathbf{n}_i^T \mathbf{R} & -\mathbf{n}_i^T \mathbf{R} [\mathbf{p}_i]_{\times} \end{bmatrix}$$

where $\mathbf{R}$ is the current rotation estimate and $[\cdot]_{\times}$ denotes the skew-symmetric operator.

Robust Cost Function

To improve robustness against outliers and degeneracies, a weighted cost function is employed. The weighting scheme combines a truncated kernel and a geometric confidence term derived from the spatial extent of each MAD-tree node:

$$\mathbf{w}_i = \left(1 - \frac{\text{bbox}_i}{b_{\min}}\right)^2 \cdot \begin{cases} 1 & |\mathbf{e}_i| \leq \rho \\ \frac{\rho}{|\mathbf{e}_i|} & |\mathbf{e}_i| > \rho \end{cases}$$

Optimization

The final pose update is computed by solving a linearized Gauss-Newton system:

$$\mathbf{H} \Delta \xi = -\mathbf{b}$$

where:

$$\mathbf{H} = \sum_i \mathbf{w}_i \mathbf{J}_i^T \mathbf{J}_i, \quad \mathbf{b} = \sum_i \mathbf{w}_i \mathbf{J}_i^T \mathbf{e}_i$$

The transformation is updated iteratively as:

$$\mathbf{T} \leftarrow \mathbf{T} \cdot \exp(\Delta \xi)$$

Parallel Implementation

The accumulation of the Hessian and gradient is parallelized across multiple threads, with per-thread buffers aggregated at each iteration to ensure thread safety and efficiency.

2.7.3 Velocity Estimation

In addition to pose estimation, the pipeline maintains an explicit estimate of the sensor velocity, which is used to improve both motion prediction and scan deskewing.

Formulation

Given a sequence of estimated poses $\{\mathbf{T}_i\}$ obtained from the ICP pipeline, the velocity is modeled as a constant twist $\mathbf{v} \in \mathbb{R}^6$ that explains the relative motion between consecutive frames:

$$\mathbf{T}_{i \rightarrow k} \approx \exp(\mathbf{v} \cdot \Delta t)$$

where Δt is the time difference between frames.

Error Model

The residual is defined separately for translation and rotation components:

$$\mathbf{e}_t = \Delta t \cdot \mathbf{v}_{lin} - (\mathbf{T}_{prev}^{-1} \mathbf{T}_{now})_t$$

$$\mathbf{e}_r = \Delta t \cdot \mathbf{v}_{ang} - \phi(\mathbf{R})$$

where $\phi(\mathbf{R})$ denotes the Euler-angle representation of the relative rotation.

Optimization

The velocity is estimated by solving a weighted least squares problem:

$$\min_{\mathbf{v}} \sum_i \mathbf{w}_i \|\mathbf{e}_i\|^2$$

where temporal weights are assigned to emphasize recent measurements:

$$\mathbf{w}_i = 1 - \frac{i}{N}$$

A robust truncation is also applied to limit the influence of outliers:

$$\mathbf{w}_i \leftarrow \min \left(\mathbf{w}_i, \frac{\tau}{\|\mathbf{e}_i\|} \right)$$

Linearization

The problem is linearized as:

$$\mathbf{H}\mathbf{v} = \mathbf{b}$$

with:

$$\mathbf{H} = \sum_i \mathbf{w}_i \mathbf{J}_i^T \mathbf{J}_i, \quad \mathbf{b} = \sum_i \mathbf{w}_i \mathbf{J}_i^T \mathbf{e}_i$$

where the Jacobian is simply:

$$\mathbf{J} = \Delta t \cdot \mathbf{I}$$

Role in the Pipeline

The estimated velocity is used to:

- Predict the initial pose for ICP optimization
- Compensate motion distortion (deskewing)
- Stabilize short-term odometry drift

Capitolo 3

Leveraging IMU for LiDAR odometry through Unscented Kalman Filter

Before analyzing the sensor fusion strategy adopted for the LiDAR-Inertial Odometry system, it is useful to briefly recall the validation framework developed for the GNSS-Inertial case, which will be discussed later in detail.

The file `example_complete.cpp` was designed as an experimental environment to analyze the behavior of the Unscented Kalman Filter under controlled conditions. Starting from a synthetic trajectory, the framework generates inertial and GNSS measurements, initializes the filter, recursively performs prediction and correction, and finally evaluates the estimation accuracy through different trajectory metrics.

This modular implementation also enabled the investigation of several initialization strategies and the influence of sensor noise, biases and initial uncertainties on the overall estimation performance. The knowledge gained from this analysis served as the basis for designing the initialization procedure adopted in the proposed LiDAR-Inertial Odometry pipeline, which is described in the following sections.

3.1 UKF: State Initialization

The initialization of the state in a nonlinear estimator such as the Unscented Kalman Filter (UKF) is a critical step, as it strongly influences both convergence speed and numerical stability. In GNSS Inertial systems, poor initialization can lead to divergence due to the nonlinear coupling between orientation, velocity, and inertial sensor biases.

The filter state is defined as:

$$\mathbf{x}_0 = \{\mathbf{R}_0, \mathbf{p}_0, \mathbf{v}_0, \mathbf{b}_a^0, \mathbf{b}_g^0\} \quad (3.1)$$

where $\mathbf{R}_0 \in SO(3)$ represents the initial orientation, $\mathbf{p}_0 \in \mathbb{R}^3$ the position, $\mathbf{v}_0 \in \mathbb{R}^3$ the velocity, and $\mathbf{b}_a^0, \mathbf{b}_g^0 \in \mathbb{R}^3$ the accelerometer and gyroscope biases respectively.

Six initialization strategies of increasing complexity have been developed. These methods aim to progressively improve observability by combining IMU preintegration and GNSS measurements.

3.1.1 INIT 1: Pure Inertial Static Initialization

This method assumes that the platform is stationary during the initial IMU samples. Under this assumption, accelerometer and gyroscope measurements are used to estimate the gravity direction and sensor biases.

The mean IMU measurements are computed as:

$$\bar{\mathbf{a}}, \quad \bar{\omega} \quad (3.2)$$

The measured gravity direction in the body frame is:

$$\mathbf{g}_{meas} = \frac{\bar{\mathbf{a}}}{\|\bar{\mathbf{a}}\|} \quad (3.3)$$

The initial rotation is obtained using Rodrigues' formula:

$$\mathbf{R}_0 = \mathbf{I} + [\mathbf{v}]_{\times} + [\mathbf{v}]_{\times}^2 \frac{1 - c}{s^2} \quad (3.4)$$

where $\mathbf{v} = \mathbf{g}_{meas} \times \mathbf{g}_{world}$, $c = \mathbf{g}_{meas} \cdot \mathbf{g}_{world}$, and $\mathbf{g}_{world} = [0, 0, -1]^T$.

Biases are initialized as:

$$\mathbf{b}_g^0 = \bar{\omega}, \quad \mathbf{b}_a^0 = \bar{\mathbf{a}} - \mathbf{R}_0^T \mathbf{g} \quad (3.5)$$

Position and velocity are initialized to zero. The covariance reflects partial observability: roll and pitch are well constrained by gravity, while yaw remains unobservable.

3.1.2 INIT 2: IMU Preintegration with GPS Constraints

This method introduces IMU preintegration between consecutive GNSS keyframes. For each segment, the preintegrated measurements are computed as:

$$\Delta \mathbf{R}, \quad \Delta \mathbf{v}, \quad \Delta \mathbf{p} \quad (3.6)$$

together with Jacobians with respect to the biases $\mathbf{J}_{b_a}, \mathbf{J}_{b_g}$.

In this formulation, an initial coarse estimate of the inertial biases is first obtained from early IMU measurements under the assumption of near-static motion. These initial values are intentionally approximate and are used only to bootstrap the preintegration process. A more accurate correction of the accelerometer bias is subsequently recovered as part of the optimization stage, where the linear system jointly refines the state.

The discrete-time motion model is:

$$\mathbf{p}_{i+1} - \mathbf{p}_i = \mathbf{v}_i \Delta t + \mathbf{R}_i \Delta \mathbf{p} + \frac{1}{2} \mathbf{g} \Delta t^2 - \mathbf{R}_i \mathbf{J}_{p_{b_a}} \Delta \mathbf{b}_a \quad (3.7)$$

A linear system of the form:

$$\mathbf{b} = \mathbf{A} \mathbf{x} \quad (3.8)$$

is constructed, where:

$$\mathbf{x} = \{\mathbf{v}_0, \dots, \mathbf{v}_{N-1}, \mathbf{g}, \Delta \mathbf{b}_a\} \quad (3.9)$$

Solving this system provides estimates for velocities, accelerometer bias correction, and gravity. The initial orientation is then propagated using $\Delta \mathbf{R}$.

3.1.3 INIT 3: Extended Window Initialization

This approach separates initialization into two stages. First, an inertial initialization (INIT 1) is used to estimate R_0 , b_a^0 , and b_g^0 under the assumption of near-static motion. Second, a longer GNSS/IMU window is used for dynamic estimation following the structure of INIT 2.

The key difference is that bias estimates from the static phase are kept fixed during preintegration.

The system becomes:

$$(\mathbf{p}_{i+1} - \mathbf{p}_i) - \mathbf{R}_i \Delta \mathbf{p} - \frac{1}{2} \mathbf{g} \Delta t^2 = \mathbf{v}_i \Delta t - \mathbf{R}_i \mathbf{J}_{p_{b_a}} \Delta \mathbf{b}_a \quad (3.10)$$

The resulting state is:

$$\mathbf{x} = \{\mathbf{v}_0, \dots, \mathbf{v}_{N-1}, \Delta \mathbf{b}_a\} \quad (3.11)$$

Yaw is optionally refined using GNSS heading estimates.

3.1.4 INIT 4: Adaptive Motion Detection

This method extends INIT 3 by introducing an automatic segmentation between static and dynamic phases.

Motion is detected when either:

$$\text{Var}(\mathbf{p}_{GNSS}) > \tau_{gps} \quad \vee \quad \|\mathbf{a}\| - \mathbf{g} > \tau_a \quad (3.12)$$

where $\tau_{gps} = 2\sigma_{gps}^2$ and τ_a is a threshold on non-gravitational acceleration.

The static segment is used for robust estimation of $\mathbf{R}_0$, $\mathbf{b}_a^0$, and $\mathbf{b}_g^0$, while the dynamic segment is used for preintegration-based estimation of velocity and bias refinement.

3.1.5 INIT 5: Cold Start Initialization

This method assumes no prior knowledge of the system state. Initialization is performed using only the first two GNSS measurements and the corresponding IMU data.

The initial velocity is estimated as:

$$\mathbf{v}_0 = \frac{\mathbf{p}_1 - \mathbf{p}_0 - \mathbf{R}_0 \Delta \mathbf{p} - \frac{1}{2} \mathbf{g} \Delta t^2}{\Delta t} \quad (3.13)$$

and refined using IMU preintegration:

$$\mathbf{v}_0 \leftarrow \mathbf{v}_0 + \mathbf{g} \Delta t + \mathbf{R}_0 \Delta \mathbf{v} \quad (3.14)$$

The orientation is propagated using $\Delta \mathbf{R}$, and the position is set to the second GNSS measurement.

3.1.6 INIT 6: Joint Estimation of Gravity and Accelerometer Bias

This method reformulates initialization as a linear estimation problem over:

$$\mathbf{x} = \{\Delta \mathbf{b}_a, \mathbf{g}\} \quad (3.15)$$

The model is expressed as:

$$\alpha_{\mathbf{k}} - \pi_{\mathbf{k}} = \mathbf{A}_k \Delta \mathbf{b}_a + \mathbf{B}_k \mathbf{g} \quad (3.16)$$

where:

- α_k is the difference of GNSS velocity averages,
- π_k is the preintegrated inertial term,
- $\mathbf{A}_k$ and $\mathbf{B}_k$ are Jacobians derived from IMU preintegration.

After solving the linear system, gravity is normalized to $\|\mathbf{g}\| = 9.81 \text{ m/s}^2$. The accelerometer bias is updated, and preintegration is recomputed accordingly to estimate velocities and propagate orientation.

3.1.7 Discussion

The six initialization strategies can be interpreted as a progressive increase in observability and robustness:

- **INIT 1:** purely inertial, limited to gravity alignment.
- **INIT 2:** introduces GNSS constraints and IMU preintegration.
- **INIT 3:** separates static and dynamic estimation phases.
- **INIT 4:** adaptive detection of motion onset.
- **INIT 5:** minimal cold start using only two GNSS keyframes.
- **INIT 6:** direct linear estimation of gravity and accelerometer bias.

The investigation of these strategies is essential to identify the most efficient initialization approach. Specifically, this analysis aims to determine the optimal starting configuration to maximize both robustness and precision for the subsequent lidar-inertial odometry problem. Overall, these methods highlight the trade-off between accuracy and computational complexity, providing a comprehensive framework for state initialization in nonlinear estimation problems.

3.2 Sensor Fusion: LiDAR Inertial Odometry

This section describes the software implementation of the proposed LiDAR-Inertial Odometry (LIO) framework. The complete framework has been developed in C++ using the Robot Operating System (ROS2) as middleware for data acquisition and management. The implementation is organized as a sequence of independent software modules, each responsible for a specific stage of the estimation process. Information flows sequentially through the pipeline, from sensor acquisition to the final state estimation, allowing each module to operate on a well-defined input and to produce a corresponding output for the subsequent stage.

The pipeline begins by loading the inertial and LiDAR measurements from a ROS2 bag file, followed by a synchronization stage that identifies the temporal interval shared by both sensors. The inertial data are then exploited to initialize the filter and estimate the initial gravity vector together with the gyroscope bias. Once initialized, the system operates iteratively on each incoming LiDAR scan. All IMU measurements acquired during the scan interval are first employed to propagate the system state through the Unscented Kalman Filter. The predicted poses are subsequently exploited to compensate for the motion distortion affecting the point cloud, producing an undistorted scan that can be reliably processed by the MAD-ICP registration algorithm. Moreover, the relative motion estimated by the UKF over the scan interval is directly employed as the initial pose prediction for the registration process, replacing the internal velocity-based prediction originally adopted by MAD-ICP.

The registration module estimates the relative motion between the current scan and the local map while simultaneously providing the Hessian matrix associated with the optimization process. The latter is exploited to estimate the observation covariance employed during the UKF correction step, enabling a probabilistic fusion between inertial propagation and LiDAR registration.

The following sections describe in detail each software module composing the proposed framework, emphasizing both the adopted data structures and the implementation choices that characterize the overall system architecture.

3.3 Sensor Fusion: Implementation details

3.3.1 Software Architecture

The proposed implementation follows a modular architecture in which each processing stage receives a well-defined input, performs a specific task and forwards its output to the subsequent module. Such organization considerably simplifies both software maintenance and future extensions, while preserving a clear correspondence between the theoretical estimation framework and its practical implementation.

Table 3.1 summarizes the main software modules composing the proposed LiDAR-Inertial Odometry pipeline.

The implementation details of each module are presented in the following subsections.

Tabella 3.1 Main software modules of the proposed LiDAR-Inertial pipeline.

Module	Input	Output
Dataset Loading	ROS2 bag	IMU and LiDAR measurements
Synchronization	Sensor measurements	Time-aligned data streams
IMU Initialization	Initial IMU samples	Initial UKF state
UKF Prediction	IMU measurements	Predicted trajectory and scan motion
Motion Compensation	Predicted trajectory, LiDAR scan	Undistorted point cloud
MAD-ICP Registration	Undistorted scan, UKF motion prediction	Relative pose and Hessian
UKF Correction	Relative pose, covariance	Updated navigation state

3.3.2 Data Structures

This section describes the main data structures adopted in the implementation of the proposed LiDAR-Inertial pipeline. These structures are designed to efficiently store and exchange information between the different processing modules while preserving temporal consistency and computational efficiency.

IMU Measurements

Inertial data are stored using the following structure:

```
struct ImuMeasurement{
    double timestamp;
    Vector6d acc_gyro;
};
```

The timestamp represents the acquisition time expressed in seconds, while the vector `acc_gyro` contains the three-axis linear acceleration followed by the three-axis angular velocity.

A sequential container is used to store all IMU measurements:

```
using ImuInputs = std::vector<ImuMeasurement>;
```

This choice ensures that all measurements are processed in chronological order, which is essential for the correct propagation of the inertial state.

LiDAR Measurements

Each LiDAR scan is represented by the following structure:

```
struct LidarMeasurement{
    double timestamp;
    double end_time;
    PointCloud::Ptr cloud;
};
```

The field `timestamp` corresponds to the acquisition time of the first point in the scan, while `end_time` represents the time instant associated with the last acquired point.

This distinction is fundamental since LiDAR scans are not instantaneous measurements. Instead, they are acquired over a finite time interval during which the sensor undergoes motion. This property is explicitly exploited in the motion compensation stage.

LiDAR measurements are stored as:

```
using LidarInputs = std::vector<LidarMeasurement>;
```

Point Cloud Representation

The Ouster LiDAR sensor provides each point with additional attributes beyond geometric coordinates. The original point structure includes intensity, ring index, reflectivity, and a per-point timestamp.

For computational efficiency, the point cloud is converted into the PCL `PointXYZINormal` format. During this conversion, the per-point timestamp is preserved by storing it in the `curvature` field:

$$\text{curvature} = t_{\text{point}}$$

This design choice allows the algorithm to access the acquisition time of each point during the deskewing phase without modifying the PCL internal structures or introducing additional memory overhead.

In addition, the original custom Ouster point type is explicitly registered using the PCL macro system to ensure compatibility with ROS message deserialization.

3.3.3 Dataset Loading and Synchronization

The first stage of the proposed pipeline consists in loading and synchronizing the inertial and LiDAR measurements stored inside a ROS2 bag dataset. This step is fundamental to ensure temporal consistency between the two sensing modalities, which is a strict requirement for any tightly-coupled LiDAR-Inertial system.

ROS2 Bag Parsing

The dataset is accessed through the `rosbag2_cpp` interface, which provides sequential access to stored messages. Two main topics are extracted during the parsing stage:

- `/os_cloud_node/imu`, containing IMU measurements;
- `/os_cloud_node/points`, containing LiDAR point cloud scans.

Each message is deserialized using the corresponding ROS serialization type and converted into the internal data structures described in the previous section.

For IMU measurements, the linear acceleration and angular velocity are extracted and stored in the `ImuMeasurement` structure. The timestamp is converted from nanoseconds to seconds in order to improve numerical stability in subsequent computations.

LiDAR Conversion

LiDAR messages are received as `sensor_msgs::PointCloud2` structures and are converted into PCL point clouds. Each point is mapped into a custom Ouster-compatible structure in order to preserve additional sensor information, such as intensity and per-point timestamp.

During this conversion, the original timestamp associated with each point is preserved by storing it in the `curvature` field of the PCL representation. This allows the motion compensation module to recover the acquisition time of each individual point without modifying the underlying PCL interface.

In addition, the timestamp corresponding to the last point of the scan is explicitly computed and stored in the `end_time` field of the LiDAR measurement structure. This value represents the temporal boundary of the scan acquisition interval.

Temporal Synchronization

After loading all measurements, a temporal synchronization step is performed in order to identify the common time interval shared by the IMU and LiDAR streams. This is necessary because the two sensors may start recording at different times.

The synchronization procedure identifies:

- the first LiDAR scan whose timestamp is greater than the first IMU measurement;
- the first IMU measurement that follows the selected LiDAR scan.

Only the overlapping portion of the dataset is retained for further processing. This ensures that every LiDAR scan is associated with a valid sequence of inertial measurements, which are later used for state propagation within the Unscented Kalman Filter.

Let t_0 and t_f denote respectively the first and last common timestamps between the two sensors:

$$t_0 = \max(t_0^{imu}, t_0^{lidar}), \quad t_f = \min(t_N^{imu}, t_M^{lidar})$$

This synchronization stage provides the temporal consistency required by the subsequent prediction and correction steps of the proposed LiDAR-Inertial estimation pipeline.

It is worth noting that IMU measurements acquired outside the strictly overlapping time interval between the two sensors are not discarded entirely. A subset of these initial inertial samples, collected before the first valid LiDAR scan, is retained and exploited exclusively during the IMU initialization stage. These measurements provide a more robust estimation of the initial gravity direction and gyroscope bias under quasi-static conditions, improving the stability of the subsequent filter initialization.

3.3.4 IMU Initialization

Before the Unscented Kalman Filter can be properly initialized, an initial estimation of the inertial biases and the gravity direction is required. This step is performed

using a short sequence of IMU measurements acquired under the assumption that the platform is either stationary or undergoing negligible motion.

The initialization process is based on the accumulation of a fixed number of IMU samples, denoted as N_{init} , which are used to estimate the statistical properties of the inertial sensors.

Mean and Covariance Estimation

The mean acceleration and angular velocity are computed using an online formulation based on Welford's algorithm, which ensures numerical stability without requiring storage of all past measurements. For each incoming IMU measurement, the estimates are updated as follows:

$$\mu_k = \mu_{k-1} + \frac{x_k - \mu_{k-1}}{k}$$

This formulation is applied independently to both accelerometer and gyroscope signals in order to obtain:

- the mean acceleration $\bar{\mathbf{a}}$,
- the mean angular velocity $\bar{\boldsymbol{\omega}}$.

In parallel, the covariance is updated incrementally in order to capture the variability of the inertial measurements during the initialization phase.

Gravity and Gyroscope Bias Estimation

Once a sufficient number of samples has been collected, the gravity direction is estimated by normalizing the mean acceleration vector:

$$\mathbf{g} = -\frac{\bar{\mathbf{a}}}{\|\bar{\mathbf{a}}\|} \cdot 9.81$$

This assumes that the dominant contribution to the accelerometer readings during initialization is due to gravity.

Similarly, the gyroscope bias is estimated as the mean angular velocity:

$$\mathbf{b}_g = \bar{\boldsymbol{\omega}}$$

These two quantities represent the initial estimates required to correctly initialize the inertial state of the system.

Initialization of the UKF State

The estimated gravity vector and gyroscope bias are used to initialize the Unscented Kalman Filter state. In addition, the initial covariance matrix is configured to reflect higher uncertainty in orientation, velocity, and position, while keeping smaller uncertainty for bias-related states.

Once the initialization phase is completed, the system transitions to the prediction and update stages of the LiDAR-Inertial pipeline.

3.3.5 State Prediction

Once the system has been initialized, the state evolution is performed through the Unscented Kalman Filter (UKF) using high-rate IMU measurements. This prediction step is responsible for propagating the navigation state between two consecutive LiDAR scans.

Let $\mathbf{x}_k$ denote the system state, which includes orientation, position, velocity, and inertial biases. The state propagation is driven by the IMU measurements of specific force and angular velocity.

IMU Integration

Between two consecutive LiDAR scans, all available IMU measurements are integrated to propagate the state forward in time. Given the asynchronous nature of the IMU, the integration is performed sequentially over each measurement interval.

For each pair of consecutive IMU samples, the input is approximated using a trapezoidal rule:

$$\mathbf{u}_k = \frac{1}{2} (\mathbf{u}_{k-1} + \mathbf{u}_k)$$

where $\mathbf{u}$ contains both accelerometer and gyroscope readings.

The corresponding time interval is computed as:

$$\Delta t_k = t_k - t_{k-1}$$

State Propagation on Manifold

The state propagation is performed on the $SO(3)$ manifold for orientation, while position and velocity are updated in $\mathbb{R}^3$. The UKF prediction step integrates the nonlinear system dynamics using the IMU inputs and the previously estimated state.

The propagation equations account for:

- angular velocity integration for orientation update,
- gravity compensation,
- bias-corrected accelerometer integration for velocity and position.

In compact form, the continuous-time dynamics can be expressed as:

$$\dot{\mathbf{R}} = \mathbf{R} \cdot (\boldsymbol{\omega} - \mathbf{b}_g)^\wedge$$

$$\dot{\mathbf{v}} = \mathbf{R} (\mathbf{a} - \mathbf{b}_a) + \mathbf{g}$$

$$\dot{\mathbf{p}} = \mathbf{v}$$

where $\mathbf{R} \in SO(3)$ denotes orientation, $\mathbf{v}$ velocity, and $\mathbf{p}$ position.

Discrete-Time Implementation

In the discrete implementation, the UKF state is propagated at each IMU arrival using numerical integration. The function `propagate()` performs the prediction step, updating both the mean state and the covariance matrix according to the process noise matrix $\mathbf{Q}$.

The process noise accounts for:

- accelerometer noise,
- gyroscope noise,
- random walk components of biases.

This prediction step provides the prior state estimate that will be later corrected using LiDAR-based measurements.

3.3.6 Motion Compensation

LiDAR scans are acquired over a finite time interval during which the sensor undergoes motion. As a consequence, each point in the point cloud is associated with a slightly different sensor pose. If not corrected, this effect introduces significant geometric distortions, commonly referred to as motion distortion or skewing.

To address this issue, a motion compensation (deskewing) procedure is applied using the high-frequency IMU-based state propagation.

Temporal Ordering of Points

Each point in the input LiDAR scan is associated with a timestamp, which is stored in the `curvature` field of the point structure. Before performing motion compensation, the point cloud is explicitly sorted in increasing temporal order:

$$t_1 \leq t_2 \leq \dots \leq t_N$$

This ensures consistency in the subsequent interpolation and state propagation steps.

Forward IMU Propagation Within the Scan

Given the initial state at the beginning of the scan, the system is propagated forward in time using the available IMU measurements. For each IMU sample, the state is integrated using the same dynamics described in the previous section.

During this process, intermediate poses are stored:

- rotation $\mathbf{R}_k$,
- position $\mathbf{p}_k$,
- velocity $\mathbf{v}_k$,
- corresponding timestamp.

These intermediate states provide a discrete approximation of the continuous trajectory of the sensor during the scan duration. In addition to motion compensation, the first and the last propagated poses are exploited to compute the relative rigid-body transformation performed by the sensor during the scan interval. This transformation is subsequently employed as the initial pose prediction for the MAD-ICP registration, replacing the internal velocity-based prediction originally adopted by the MAD-ICP solo-pipeline.

Backward Point Compensation

Each LiDAR point is compensated with respect to the sensor pose at the end of the scan. Let $\mathbf{p}_i$ denote a point acquired at time t_i and expressed in the LiDAR reference frame. The goal of motion compensation is to remove the distortion induced by the sensor motion during the scan interval by expressing all points in a common reference frame.

Let t_0 and t_f denote respectively the beginning and the end of the LiDAR scan. The estimated poses of the IMU are available at discrete timestamps within this interval through the propagation step described in the previous section.

For each point, the relative motion between its acquisition time t_i and the scan end time t_f is computed by combining rotational and translational components derived from IMU integration.

The rotation at time t_i is obtained by integrating the angular velocity:

$$\mathbf{R}_i = \mathbf{R}_k \cdot \exp((\boldsymbol{\omega}_k - \mathbf{b}_g)(t_i - t_k))$$

where $\mathbf{R}_k$ is the nearest propagated IMU orientation and $\exp(\cdot)$ denotes the exponential map on $SO(3)$.

The translational offset between t_i and t_f is modeled using a constant-acceleration approximation:

$$\mathbf{T}_{i \rightarrow f} = \mathbf{p}_k + \mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{a}_k \Delta t^2 - \mathbf{p}_f$$

where $\mathbf{p}_k$, $\mathbf{v}_k$ and $\mathbf{a}_k$ are respectively the position, velocity and linear acceleration estimated from IMU propagation, and $\Delta t = t_i - t_k$.

The final compensated point is obtained by chaining LiDAR-to-IMU extrinsics and relative IMU motion:

$$\mathbf{p}_i^{corr} = \mathbf{R}_{LI}^\top \left[\mathbf{R}_f^\top (\mathbf{R}_i (\mathbf{R}_{LI} \mathbf{p}_i + \mathbf{t}_{LI}) + \mathbf{T}_{i \rightarrow f}) - \mathbf{t}_{LI} \right]$$

where $\mathbf{R}_{LI}$ and $\mathbf{t}_{LI}$ represent the rigid transformation between LiDAR and IMU frames, and $\mathbf{R}_f$ is the orientation at the end of the scan.

This formulation ensures that all points are expressed in a consistent reference frame corresponding to the end-of-scan pose, effectively removing motion-induced distortions while preserving the original geometric structure of the environment.

Practical Considerations on Temporal Consistency

In practical scenarios, the end time of a LiDAR scan does not always coincide exactly with the timestamp of the last available IMU propagation state. This misalignment

may lead to situations in which a subset of LiDAR points is acquired after the last valid IMU pose within the scan interval.

In order to maintain temporal consistency between the LiDAR compensation and the IMU-based state propagation, such points are explicitly discarded. This ensures that the final reference pose of the scan corresponds exactly to the last propagated IMU state, avoiding extrapolation beyond the available inertial information.

Although this strategy reduces the number of usable points within a scan, it significantly improves the robustness of the motion compensation process. In particular, it prevents the introduction of geometric artifacts that may arise from unreliable extrapolation of the IMU trajectory at the end of the scan.

Empirically, this design choice has been observed to improve the overall consistency of the reconstructed trajectory. This trade-off can be interpreted as favoring accuracy over density: it is preferable to use a reduced set of geometrically consistent points rather than a larger but partially distorted point cloud.

Resulting Undistorted Point Cloud

After compensation, all points are expressed in a common reference frame corresponding to the end of the LiDAR scan (and also to the last IMU pose). The resulting undistorted point cloud can then be reliably used by the registration module without introducing geometric inconsistencies due to motion distortion.

This step is crucial for ensuring the accuracy of the subsequent LiDAR-based odometry estimation, as it enforces a rigid-body assumption on the corrected point cloud.

Besides producing an undistorted scan suitable for registration, the UKF propagation also provides an estimate of the sensor motion during the scan interval. This relative motion is forwarded to the registration module to initialize the scan-to-map optimization, establishing a tighter coupling between inertial propagation and LiDAR registration.

3.3.7 MAD-ICP Integration

The LiDAR odometry module is responsible for estimating the relative motion between consecutive scans using a registration-based approach. In the proposed framework, this role is performed by the MAD-ICP pipeline, which aligns the current undistorted point cloud with a local geometric model.

Unlike traditional ICP formulations that directly produce a deterministic pose estimate, the MAD-ICP module is used here as a measurement generator for the probabilistic state estimator.

UKF-Based Registration Initialization

The original MAD-ICP implementation estimates the initial registration guess through an internal velocity estimator, which predicts the sensor motion assuming a locally constant body velocity inferred from previous odometry estimates.

In the proposed framework, this prediction is replaced by the relative motion directly estimated by the UKF during the IMU propagation stage. Let $\mathbf{T}_0$ and $\mathbf{T}_f$

denote the predicted sensor poses respectively at the beginning and at the end of the current LiDAR scan. The relative motion over the scan interval is computed as

$$\Delta \mathbf{T}_{UKF} = \mathbf{T}_0^{-1} \mathbf{T}_f.$$

The predicted pose used to initialize the MAD-ICP optimization is then obtained as

$$\mathbf{T}_{pred} = \mathbf{T}_{k-1} \Delta \mathbf{T}_{UKF},$$

where $\mathbf{T}_{k-1}$ denotes the scan pose estimated during the previous iteration.

Compared to the original implementation, this modification removes the dependency on the internal velocity estimator and provides an initialization that is fully consistent with the inertial prediction already computed by the UKF. As a consequence, the registration process starts from a more informed initial estimate while preserving the original optimization strategy of MAD-ICP.

Relative Pose Estimation

Given an undistorted point cloud together with the relative motion estimated by the UKF over the scan interval, the registration pipeline computes the relative transformation $\mathbf{Z}$ between the current scan and the reference map. This transformation belongs to the Lie group $SE(3)$ and is composed of a rotation and a translation:

$$\mathbf{Z} = \begin{bmatrix} \mathbf{R}_z & \mathbf{t}_z \\ \mathbf{0} & 1 \end{bmatrix} \in SE(3)$$

The transformation is obtained by minimizing a point-to-distribution alignment criterion within the MAD-ICP framework, which iteratively refines the alignment between the scan and the local model.

Measurement Representation: Update on Manifold

In the proposed implementation, the LiDAR measurement is directly provided to the Unscented Kalman Filter as a full rigid-body transformation $\mathbf{Z} \in SE(3)$, without an explicit projection onto a Euclidean tangent space.

The filter has been extended to operate on manifold-valued states, allowing both the state and the measurement to be treated consistently within the Lie group structure. In particular, the internal update equations of the UKF have been modified to correctly handle non-Euclidean quantities, avoiding inconsistent operations such as direct addition or subtraction of poses in $SE(3)$.

This modification was introduced after observing numerical inconsistencies in the standard formulation of the filter when applied to manifold-valued states. In the original implementation, operations such as computing the innovation term required ad-hoc projections (e.g., $\log(\cdot)$ mappings), which led to inconsistencies in the computation of the mean measurement $\bar{\mathbf{z}}$.

By reformulating the filter to operate directly on the manifold, the computation of the innovation and the update step are performed using Lie group consistent

operations. This ensures that all statistical operations respect the geometry of the state space.

For debugging and analysis purposes, the relative transformation estimated by the MAD-ICP module is also expressed in tangent coordinates:

$$\mathbf{z}_{tangent} = \begin{bmatrix} \mathbf{t}_z \\ \log(\mathbf{R}_z) \end{bmatrix}$$

However, this representation is not used within the UKF update step, but only for logging and offline evaluation of the estimated relative motion.

Measurement Covariance Estimation

A key aspect of the proposed approach is the estimation of the measurement uncertainty associated with the LiDAR registration. Instead of assuming a fixed covariance, the uncertainty is derived directly from the optimization process.

The MAD-ICP module provides a Hessian matrix $\mathbf{H}$, which encodes the curvature of the cost function at the solution. Under a Gaussian approximation, the measurement covariance is computed as:

$$\mathbf{R} = \mathbf{H}^{-1}$$

In practice, a small regularization term is added to ensure numerical stability:

$$\mathbf{H} \leftarrow \mathbf{H} + \epsilon \mathbf{I}$$

This formulation allows the filter to adaptively weight LiDAR measurements depending on the local geometric structure of the environment.

Role within the Estimation Pipeline

The output of the MAD-ICP module consists of the measurement vector $\mathbf{Z}$ and its associated covariance $\mathbf{R}$. These quantities are then directly used in the UKF correction step to update the navigation state.

This design enables a tight coupling between geometric registration and probabilistic filtering, where the reliability of the LiDAR observation is directly inferred from the registration process itself.

3.3.8 UKF Correction

The correction step of the proposed framework fuses the LiDAR-based relative pose estimate with the inertially propagated state within the Unscented Kalman Filter (UKF). This stage is responsible for correcting drift accumulated during the prediction phase and for enforcing consistency with the geometric information provided by the LiDAR registration module.

Measurement Model

The measurement provided by the MAD-ICP module consists of a relative rigid-body transformation $\mathbf{Z} \in SE(3)$ between the current scan and the reference frame. This transformation is interpreted as a direct observation of the system state evolution over the LiDAR scan interval.

The innovation is computed by comparing the predicted state with the measured transformation, taking into account the non-Euclidean nature of the state space.

Innovation on Manifold

Let $\hat{\mathbf{X}}$ denote the predicted state and $\mathbf{Z}$ the LiDAR measurement. The innovation is defined using group-consistent operations on $SE(3)$:

$$\boldsymbol{\delta} = \log(\hat{\mathbf{X}}^{-1} \cdot \mathbf{Z})$$

where $\log(\cdot)$ denotes the logarithmic map from $SE(3)$ to its tangent space $R(3)$.

This formulation ensures that the residual is expressed in a minimal vector space while preserving the geometric consistency of the underlying state representation.

Kalman Gain Computation

The Unscented Kalman Filter propagates sigma points through the nonlinear measurement model to compute the predicted measurement statistics. The Kalman gain is then obtained as:

$$\mathbf{K} = \mathbf{P}_{xz} \mathbf{P}_{zz}^{-1}$$

where $\mathbf{P}_{xz}$ represents the cross-covariance between state and measurement, and $\mathbf{P}_{zz}$ is the measurement covariance derived from the MAD-ICP Hessian.

State Update

The state correction is applied on the manifold using a retraction operator. The updated state is computed as:

$$\mathbf{X}^+ = \mathbf{X}^- \oplus (\mathbf{K}\boldsymbol{\delta})$$

where $\oplus$ denotes the manifold retraction mapping the correction term back onto $SE(3)$.

All ancillary state components, including velocity and inertial biases, are updated consistently within the same framework.

Covariance Update

The state covariance is updated according to the standard UKF formulation:

$$\mathbf{P}^+ = \mathbf{P}^- - \mathbf{K} \mathbf{P}_{zz} \mathbf{K}^\top$$

This step reduces the uncertainty of the state estimate by incorporating the information provided by the LiDAR measurement.

Role in the Overall Pipeline

The UKF correction step completes the sensor fusion cycle by combining high-rate inertial propagation with geometrically constrained LiDAR observations. This results in a drift-corrected estimate of the full navigation state, which is then used as the prior for the next prediction phase.

3.3.9 Complete Pipeline

This final section summarizes the overall structure of the proposed LiDAR-Inertial odometry framework. The system integrates all previously described modules into a unified estimation pipeline that processes asynchronous IMU and LiDAR measurements in a sequential manner.

The algorithm operates iteratively over the dataset, processing each LiDAR scan together with the corresponding inertial measurements.

Algorithm Overview

The complete pipeline can be summarized in Algorithm 1:

Algorithm 1 Proposed LiDAR-Inertial Odometry Pipeline

- 1: Load IMU and LiDAR measurements
 - 2: Synchronize data streams
 - 3: Initialize IMU bias and gravity
 - 4: Initialize UKF state
 - 5: **for** each LiDAR scan **do**
 - 6: Collect IMU measurements in scan interval
 - 7: Propagate state using IMU (UKF prediction)
 - 8: Perform motion compensation (deskewing) of LiDAR point cloud using propagated poses
 - 9: Compute relative pose $\mathbf{Z}$ using MAD-ICP
 - 10: Compute covariance $\mathbf{R}$ from Hessian
 - 11: Update UKF state on $SE(3)$ manifold
 - 12: **end for**
 - 13: Return estimated trajectory
-

Discussion

The proposed pipeline tightly integrates inertial and LiDAR information within a single probabilistic framework. The IMU provides high-frequency motion prediction, while the LiDAR measurements provide geometrically constrained corrections.

The explicit use of manifold-aware operations ensures consistency when dealing with rotational quantities, while the adaptive covariance estimation from the MAD-ICP Hessian allows the filter to dynamically weight LiDAR observations based on environmental structure.

Overall, the system results in a robust LiDAR-Inertial odometry framework capable of operating in challenging environments with significant motion dynamics and varying geometric constraints.

Capitolo 4

Experiments and Results

4.1 Preliminary Experiments: GNSS Inertial Odometry with UKF

As previously announced, the file `example_complete.cpp` provides a complete implementation of a GNSS-Inertial Odometry pipeline based on the Unscented Kalman Filter (UKF). This example employs simulated IMU and GNSS measurements in order to evaluate the estimation performance under controlled conditions.

The program generates synthetic trajectories together with the corresponding inertial measurements, corrupts the IMU data with configurable levels of noise and bias, simulates noisy GNSS observations, and estimates the platform trajectory through recursive prediction and correction steps.

The complete processing pipeline can be summarized as follows:

Algorithm 2 GNSS-Inertial Odometry Pipeline

- 1: Generate synthetic trajectory
 - 2: Compute ideal IMU measurements from the trajectory
 - 3: Add noise and biases to IMU measures
 - 4: Simulate GNSS observations
 - 5: Initialize UKF state and covariance
 - 6: **for** each IMU sample **do**
 - 7: Propagate the state using inertial measurements
 - 8: **if** a GNSS measurement is available **then**
 - 9: Perform UKF measurement update
 - 10: **end if**
 - 11: Store estimated state
 - 12: **end for**
 - 13: Compute trajectory evaluation metrics
 - 14: Export trajectories and biases
-

This implementation was mainly designed as an experimental environment for analyzing the behavior of the UKF under different sensor noise conditions, bias magnitudes and trajectory geometries.

4.1.1 Synthetic Dataset Generation

Ground-Truth Trajectory Generation

The experimental evaluation is entirely based on synthetic trajectories generated offline. This approach allows the exact ground-truth state of the platform to be known at every instant, making it possible to quantitatively evaluate the estimation accuracy of the proposed filter.

Four different trajectory geometries are available within the implementation:

- Circular trajectory;
- Wave trajectory;
- Figure-eight trajectory;
- Spiral trajectory.

The trajectory type is selected through a command-line parameter, while its spatial amplitude can be independently adjusted. Given the desired trajectory duration T and the IMU sampling frequency f_{imu} , the complete sequence is discretized into

$$N = T \cdot f_{imu}$$

samples, separated by the constant integration interval

$$\Delta t = \frac{1}{f_{imu}}.$$

The platform positions are analytically generated according to the selected trajectory model. For example, the circular trajectory is obtained as

$$\begin{aligned} x(t) &= r \sin\left(2\pi \frac{t}{T}\right), \\ y(t) &= r \cos\left(2\pi \frac{t}{T}\right), \end{aligned}$$

where r denotes the trajectory radius.

Similarly, the remaining trajectories are generated through different analytical expressions, allowing the filter to be tested under various motion patterns characterized by different curvatures and acceleration profiles.

Computation of Velocities and Accelerations

Once the ground-truth positions have been generated, the corresponding linear velocities and accelerations are numerically computed through finite differences.

The velocity at each sampling instant is obtained as

$$\mathbf{v}_k = \frac{\mathbf{p}_k - \mathbf{p}_{k-1}}{\Delta t},$$

while the linear acceleration is computed as

$$\mathbf{a}_k = \frac{\mathbf{v}_k - \mathbf{v}_{k-1}}{\Delta t}.$$

Although the trajectory is analytically defined, this numerical differentiation reproduces the information that would normally be available from discrete measurements, providing a realistic input for the inertial simulation.

The resulting position, velocity and acceleration sequences constitute the complete ground-truth motion used throughout the remaining stages of the simulation.

Simulation of Ideal IMU Measurements

The inertial measurements are generated directly from the ground-truth trajectory. At each time step, the linear acceleration expressed in the world frame is converted into the IMU reference frame before being used as filter input.

Since an accelerometer measures the specific force rather than the absolute acceleration, the gravity vector must first be removed. The simulated accelerometer measurement is therefore computed as

$$\mathbf{a}_{imu} = \mathbf{R}^T (\mathbf{a} - \mathbf{g}),$$

where $\mathbf{R}$ denotes the current body orientation, $\mathbf{a}$ is the linear acceleration expressed in the world frame and $\mathbf{g}$ is the gravity vector.

The resulting quantity corresponds to the ideal accelerometer measurement that would be observed by an inertial sensor moving along the prescribed trajectory.

At the current stage of development, the gyroscope measurements are intentionally neglected and are initialized to zero. Although the implementation already contains the necessary infrastructure for generating angular velocities from the trajectory, this functionality has been left disabled since the experimental analysis presented in this work mainly focuses on translational motion.

Noise and Bias Injection

In order to evaluate the robustness of the proposed estimator under different operating conditions, three different sets of inertial measurements are generated.

The first dataset contains ideal IMU measurements corrupted only by zero-mean Gaussian noise,

$$\mathbf{u}_{noise} = \mathbf{u}_{gt} + \mathbf{n},$$

where $\mathbf{u}_{gt}$ denotes the ideal inertial measurement and $\mathbf{n}$ represents white Gaussian noise.

The second dataset contains only slowly varying sensor biases,

$$\mathbf{u}_{bias} = \mathbf{u}_{gt} + \mathbf{b},$$

where the bias follows a random walk process,

$$\mathbf{b}_k = \mathbf{b}_{k-1} + \mathbf{w},$$

with $\mathbf{w}$ being zero-mean Gaussian noise.

Finally, the third dataset combines both effects,

$$\mathbf{u}_{dirty} = \mathbf{u}_{gt} + \mathbf{b} + \mathbf{n},$$

thus representing a more realistic inertial sensor affected simultaneously by stochastic noise and slowly drifting biases.

4.1.2 Unscented Kalman Filter Initialization

Before the estimation process, the simulation parameters together with the IMU and GNSS sensor characteristics are configured. The trajectory geometry, duration and amplitude can be selected through command-line arguments, while the IMU and GNSS frequencies are fixed to 100 Hz and 5 Hz, respectively.

The process noise covariance is initialized as

$$\mathbf{Q} = \text{diag} \left(\sigma_a^2, \sigma_\omega^2, \sigma_{ba}^2, \sigma_{bg}^2 \right),$$

where the variances model accelerometer noise, gyroscope noise and the random-walk evolution of the corresponding biases.

The filter is initialized from the first ground-truth pose. Two state representations are considered: the basic `NavState`, containing orientation, velocity and position, and the extended `NavStateExtended`, which additionally estimates accelerometer and gyroscope biases. To evaluate convergence, a controlled initialization error consisting of a 3° yaw offset and a 0.3 m position error is introduced.

Finally, the initial state covariance and the UKF sigma-point weights are configured according to the selected state representation, allowing the recursive prediction-correction process to begin.

4.1.3 Unscented Kalman Filter Prediction

The prediction step propagates the navigation state using the incoming IMU measurements through the nonlinear motion model described in the previous chapter. At each iteration, the filter is updated by calling

```
ukf.propagate(ukf_state, inputs.at(i-1), Q_inputs, dt);
```

where the current state is propagated using the IMU input, the process noise covariance and the integration interval. For the extended state representation, the covariance associated with the IMU biases is additionally increased according to a random-walk model after each propagation step.

4.1.4 GNSS Measurement Update

Whenever a GNSS observation is available, the predicted state is corrected through the UKF measurement update:

```
if(zs_gps.count(i) > 0){
    obs.meas() = zs_gps.at(i);
    ukf.update(ukf_state, obs, R);
}
```

The measurement covariance is defined as

$$\mathbf{R} = \sigma_{gps}^2 \mathbf{I}_3,$$

where σ_{gps} denotes the GNSS standard deviation. The correction compensates for the drift accumulated during inertial propagation, resulting in an estimation process that alternates high-rate IMU predictions with lower-rate GNSS updates.

4.1.5 Trajectory Evaluation Metrics

The estimated trajectories produced by the different filter configurations are quantitatively evaluated by comparing them with the corresponding ground-truth trajectory.

Since different sources of estimation error may affect the reconstructed trajectory, several complementary evaluation metrics are employed. In particular, both global and local trajectory errors are analyzed in order to separately evaluate the overall reconstruction accuracy and the accumulated drift during the estimation process.

The proposed framework computes four performance indicators:

- Absolute Trajectory Error (ATE);
- Aligned Absolute Trajectory Error (ATE₂);
- Relative Pose Error in translation (RPE-T);
- Relative Pose Error in rotation (RPE-R).

The implementation of these metrics allows a comprehensive comparison between the different state representations and sensor configurations adopted throughout the experimental evaluation.

Absolute Trajectory Error

The Absolute Trajectory Error (ATE) measures the global discrepancy between the estimated trajectory and the ground-truth trajectory.

Given the estimated position $\hat{\mathbf{p}}_i$ and the corresponding ground-truth position $\mathbf{p}_i$, the metric is defined as

$$ATE = \sqrt{\frac{1}{N} \sum_{i=1}^N \|\mathbf{p}_i - \hat{\mathbf{p}}_i\|^2}.$$

Unlike alignment-based metrics, this formulation preserves any initial pose error introduced during the filter initialization. Consequently, ATE evaluates the overall localization accuracy, including initialization offsets.

Since GNSS measurements provide absolute position information in a global reference frame, this metric is particularly suitable for the GNSS-Inertial Odometry framework presented in this chapter. In this context, ATE directly reflects the capability of the estimator to recover the absolute trajectory while accounting for both initialization errors and the accuracy of the GNSS observations.

Aligned Absolute Trajectory Error

Although ATE provides an overall estimation error, it is strongly influenced by the initial pose alignment.

For this reason, an additional metric is introduced. Before computing the trajectory error, the estimated trajectory is rigidly aligned to the ground truth using the Umeyama algorithm. This procedure estimates the optimal rotation and translation that minimize the least-squares error between the two trajectories.

In practice, this aligned formulation is the standard way in which Absolute Trajectory Error is reported in the literature, as it removes the dependency on the initial global frame alignment and focuses on the intrinsic consistency of the estimated motion.

After alignment, the Absolute Trajectory Error becomes

$$ATE_2 = \sqrt{\frac{1}{N} \sum_i \|\mathbf{p}_i - (\mathbf{R}\hat{\mathbf{p}}_i + \mathbf{t})\|^2}.$$

This metric evaluates the intrinsic reconstruction capability of the filter independently of the initial pose offset.

The rigid transformation is computed through a manual implementation of the Umeyama algorithm based on Singular Value Decomposition (SVD). The resulting rotation and translation are subsequently used to align the estimated trajectory before evaluating the reconstruction error.

Relative Pose Error

While the Absolute Trajectory Error evaluates the global reconstruction quality, it does not directly measure the local drift accumulated during navigation.

To characterize this aspect, the Relative Pose Error is computed between pairs of poses separated by a fixed temporal interval.

Two complementary quantities are evaluated:

- translation error (RPE-T);
- rotation error (RPE-R).

The translational Relative Pose Error measures the difference between the estimated and ground-truth relative displacement.

The error is computed from the relative transformation

$$\mathbf{E} = \mathbf{T}_{gt}^{-1} \mathbf{T}_{gt}^+ \left(\mathbf{T}_{est}^{-1} \mathbf{T}_{est}^+ \right)^{-1},$$

where the translational component of $\mathbf{E}$ is used to compute the root mean squared error over the entire trajectory.

Similarly, the rotational Relative Pose Error evaluates the discrepancy between consecutive orientation estimates.

The rotational error is extracted from the residual rotation matrix

$$\mathbf{E}_R = \mathbf{R}_{gt} \mathbf{R}_{est}^T,$$

whose angular magnitude is computed through the angle-axis representation.

This metric provides an effective measure of the orientation drift accumulated by the navigation filter.

The simultaneous use of global and local trajectory metrics provides a comprehensive assessment of the navigation performance. While ATE and ATE₂ quantify the overall reconstruction accuracy, the Relative Pose Errors evaluate the short-term consistency of the estimated motion. Together, these indicators allow the different filter configurations to be compared under identical experimental conditions.

4.1.6 Output Generation

The final stage of the proposed framework consists of exporting the estimated trajectories and the quantities generated during the estimation process. These outputs are used for both qualitative visualization and quantitative performance evaluation.

Rather than limiting the execution to the computation of the navigation solution, the framework automatically stores all relevant information into text files, enabling offline analysis, trajectory comparison and post-processing. This design choice facilitates the reproducibility of the experiments and simplifies the evaluation of different filter configurations.

Trajectory Export

At the end of the estimation process, the framework exports both the ground-truth trajectory and the trajectories estimated by each filter configuration.

The generated files contain the Cartesian coordinates of the estimated positions for every timestamp:

```
std::ofstream file("traj_est.txt");

for(const auto& s : estimates){
    file << s.position().x() << " "
    << s.position().y() << " "
    << s.position().z() << "\n";
}
```

Similar files are generated for all the considered filter configurations, including the standard navigation state, the extended state with bias estimation, and the corresponding noisy sensor scenarios.

Tabella 4.1 Trajectory files generated by the framework.

File	Description
traj_gt.txt	Ground-truth trajectory
traj_est.txt	Estimated trajectory (noise only)
traj_est_bias.txt	Estimated trajectory with biased IMU
traj_est_dirt.txt	Estimated trajectory with noisy and biased IMU
traj_est_ext_bias.txt	Extended-state trajectory (bias estimation)
traj_est_ext_dirt.txt	Extended-state trajectory (noise + bias estimation)

```

file << b_acc.x()
      << b_acc.y()
      << b_acc.z()
      << b_gyro.x()
      << b_gyro.y()
      << b_gyro.z();

```

Experimental Reproducibility

In addition to the estimated trajectories, the framework exports all the information required to reproduce the conducted experiments.

The generated datasets include:

- simulated IMU measurements;
- simulated GNSS observations;
- ground-truth states;
- estimated trajectories;
- estimated IMU biases.

By storing every intermediate result, the complete estimation pipeline can be reproduced offline without regenerating the synthetic dataset. Furthermore, the exported files can be directly employed by external visualization tools or numerical analysis scripts.

The `example_complete.cpp` program therefore represents a complete validation framework for the proposed GNSS-Inertial navigation system. Starting from a synthetic trajectory, the software generates realistic sensor measurements, initializes the navigation filter, performs recursive state estimation, evaluates the reconstruction accuracy through multiple performance metrics, and finally exports all relevant results for subsequent analysis.

The modular organization of the implementation also facilitates the extension of the framework with additional sensor modalities, making it a suitable starting point for the LiDAR-Inertial odometry system presented in the following sections.

4.1.7 Experimental Setup: NavState VS NavStateExtended

The first set of experiments compares two different UKF state representations. The first one, referred to as *NavState*, estimates only the navigation variables, namely **orientation**, **velocity** and **position**. The second one, *NavStateExtended*, augments the state with **accelerometer** and **gyroscope biases**, allowing the filter to compensate for slowly varying sensor offsets during the estimation process.

The experiments are performed using synthetic IMU measurements generated according to the *Kalibr IMU noise model*.

The performance of both filters is evaluated under different operating conditions by varying the accelerometer and gyroscope noise, the corresponding bias magnitudes, the trajectory geometry (circular, sinusoidal, figure-eight and spiral), and the trajectory scale.

4.2 Preliminary Experiments: Results

4.2.1 Analysis of Filter Performance under Varying Bias Multipliers

The first set of experiments evaluates the performance of the proposed filtering approaches, namely *NavState* and *NavStateExtended*, by assessing key metrics—specifically ATE, ATE2, RPET, and RPER—across a broad spectrum of IMU bias multipliers. These multipliers were systematically varied within the range $[0, 100]$, representing increasing levels of sensor uncertainty.

For each bias configuration, a corresponding trajectory estimation was performed, and the evaluation metrics were logged to characterize the filters' sensitivity to noise. The resulting trends are illustrated in figures 4.1a, 4.1b, 4.1c and 4.1d.

As depicted in the plots for ATE and ATE2, the estimation error for the *NavState* filter exhibits a linear increase with a constant positive slope as the bias multiplier rises. In contrast, the *NavStateExtended* filter demonstrates superior robustness; its estimation error remains substantially stable despite the increasing noise levels. This behavior confirms the enhanced capability of the extended formulation to mitigate the impact of sensor inaccuracies.

To provide a qualitative assessment of these performance differences, figures 4.2, 4.3, 4.4, 4.5 and 4.6 showcases a comparison of estimated trajectories from *NavState* (yellow) and *NavStateExtended* (light-blue), sampled at critical bias values across the tested range.

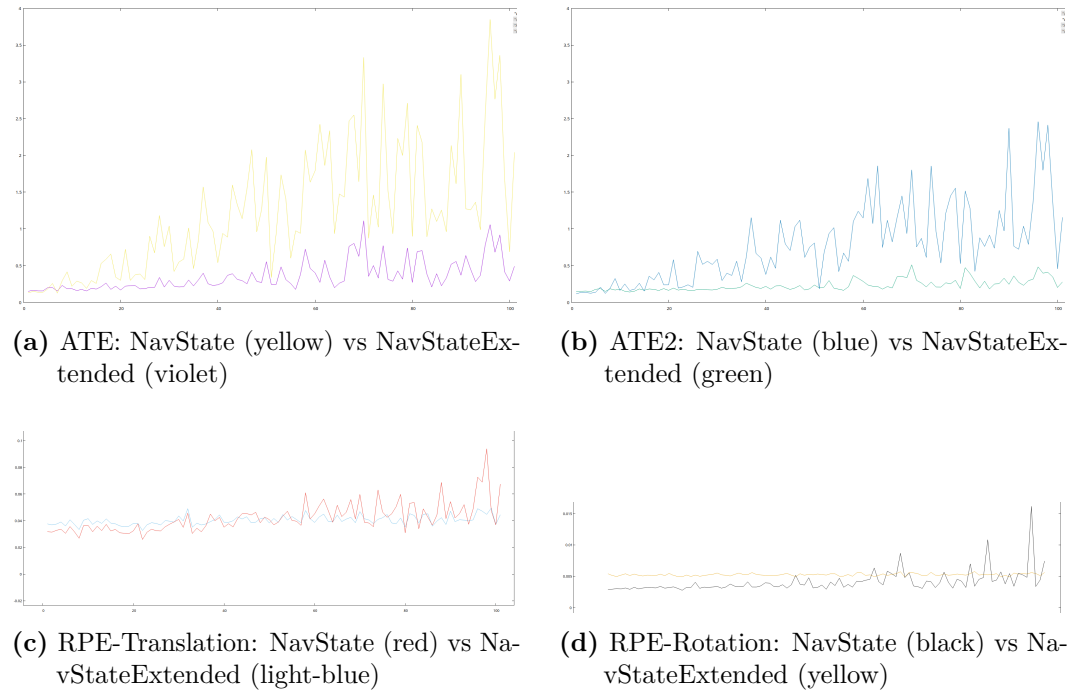


Figure 4.1 Performance comparison between NavState and NavStateExtended.

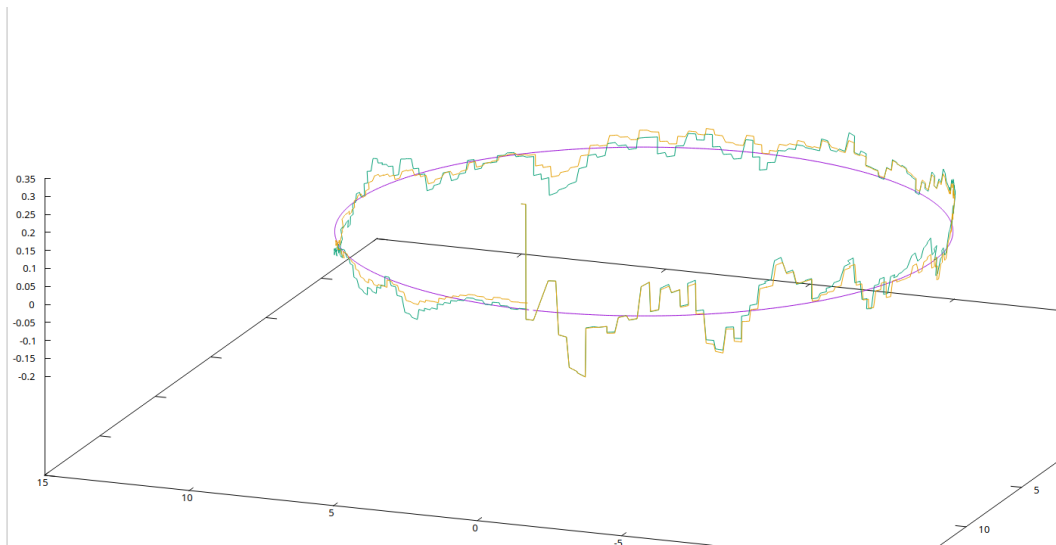


Figure 4.2 bias = 1; NavState (yellow) NavStateExtended (light-blue)

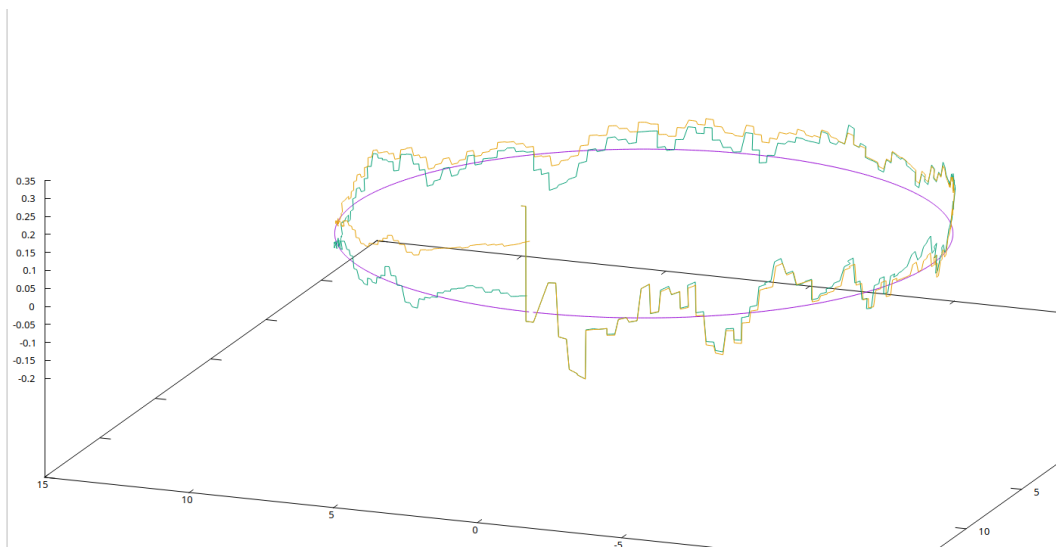


Figure 4.3 bias = 5; NavState (yellow) and NavStateExtended (light-blue)

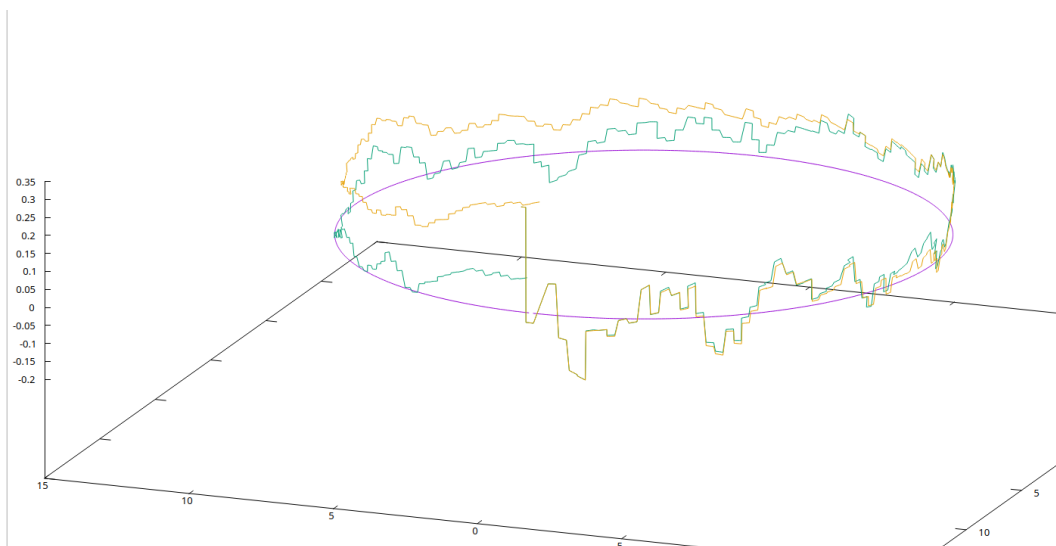


Figure 4.4 bias = 10; NavState (yellow) and NavStateExtended (light-blue)

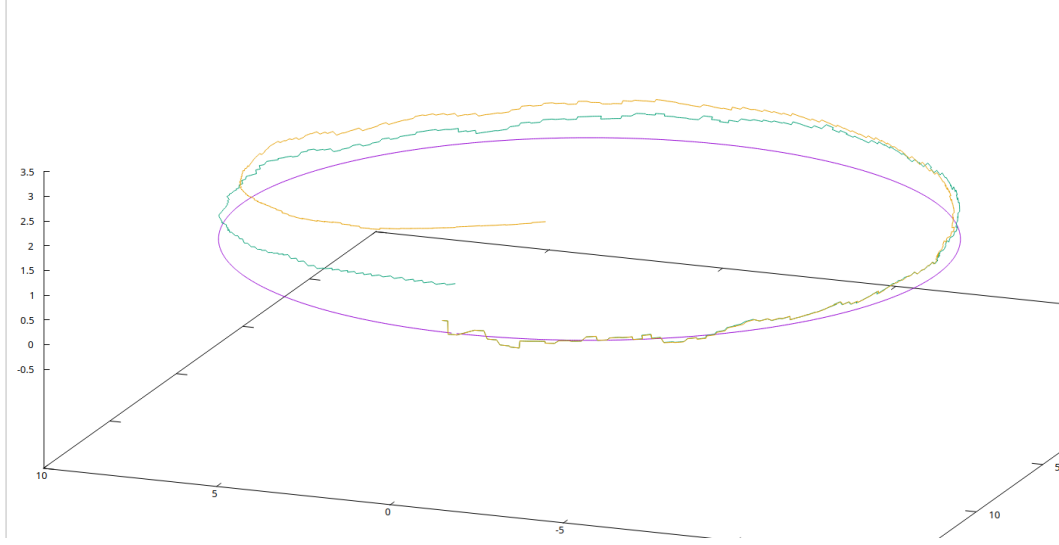


Figure 4.5 bias = 100; NavState (yellow) and NavStateExtended (light-blue)

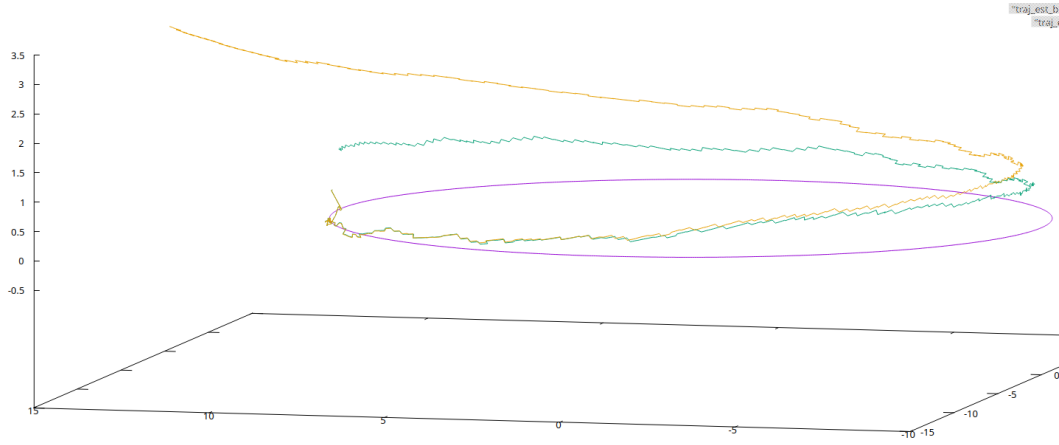


Figure 4.6 bias = 100, another point of view

4.2.2 Analysis of Filter Robustness under Varying Trajectory Amplitude

A subsequent experimental analysis was conducted to evaluate the performance of the *NavState* and *NavStateExtended* filters by varying the trajectory amplitude within the range $[0.5, 100]$. In these scenarios, the IMU noise parameters (sigmas) were scaled by a factor of 10, a configuration representative of low-cost commercial IMU sensors.

Figures 4.7a, 4.7b, 4.7c and 4.7d illustrates the resulting trends for the ATE, ATE2, RPET, and RPER metrics. Regarding the ATE and ATE2 metric, *NavStateExtended* consistently exhibits lower estimation errors compared to *NavState*, confirming its superior global trajectory tracking capability.

A distinctive advantage of the *NavStateExtended* formulation is its ability to significantly mitigate drift accumulation over long distances. Since uncorrected drift errors compound over time, the extended formulation proves particularly effective in maintaining trajectory accuracy as the travelled distance increases. To further illustrate these characteristics, figures 4.8, 4.9, 4.10 and 4.11 presents a qualitative comparison of trajectories estimated by both filters, sampled at significant amplitude levels and displacement vectors across the tested range.

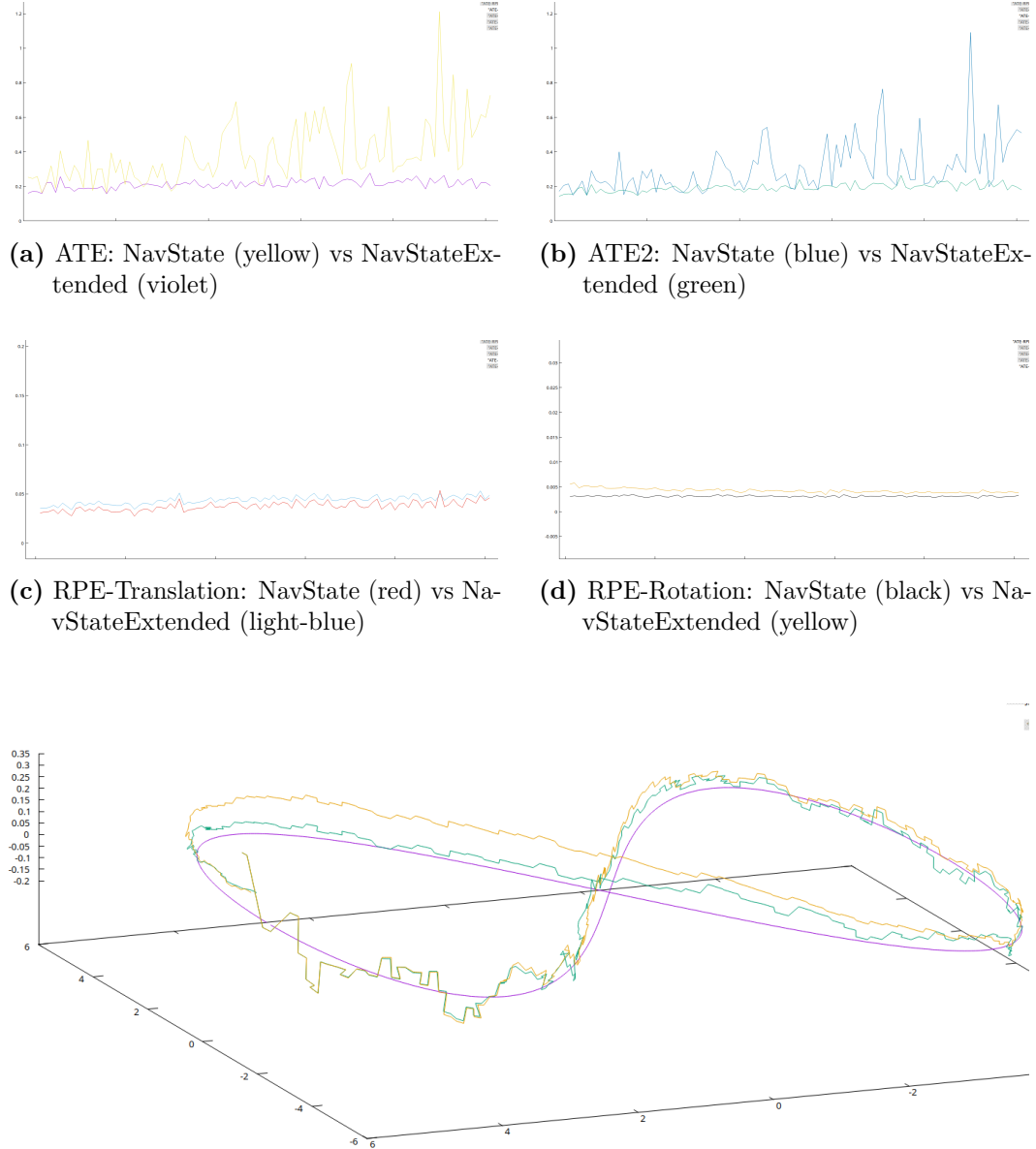


Figure 4.8 amplitude = 5; NavState (yellow) NavStateExtended (light-blue)

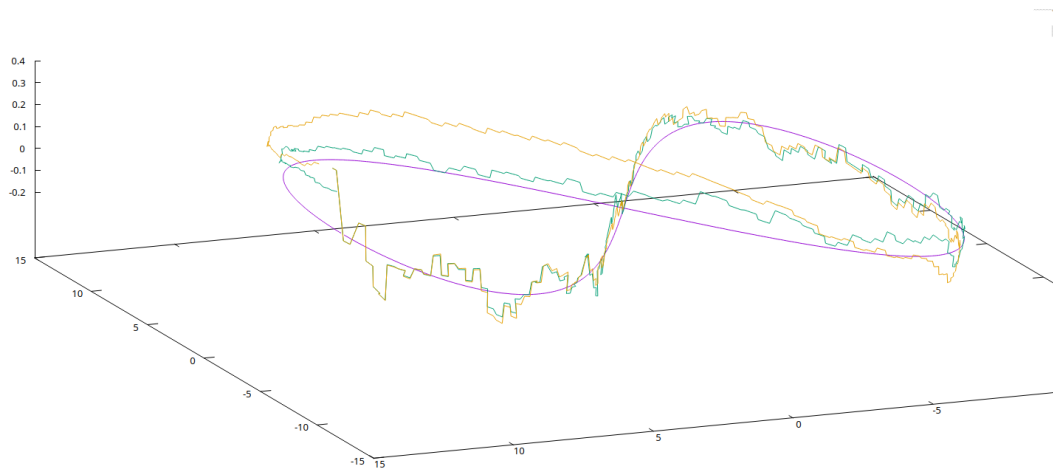


Figure 4.9 amplitude = 10; NavState (yellow) and NavStateExtended (light-blue)

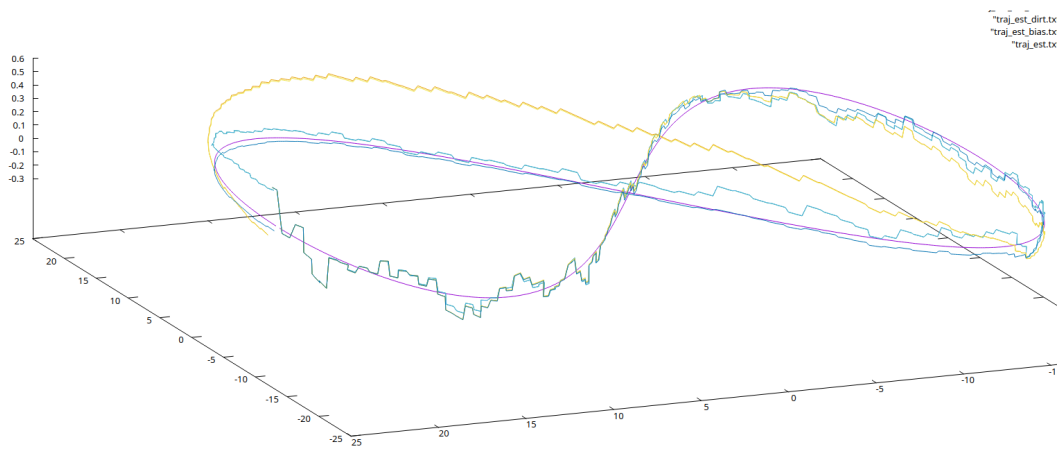


Figure 4.10 amplitude = 20; NavState (yellow) and NavStateExtended (light-blue)

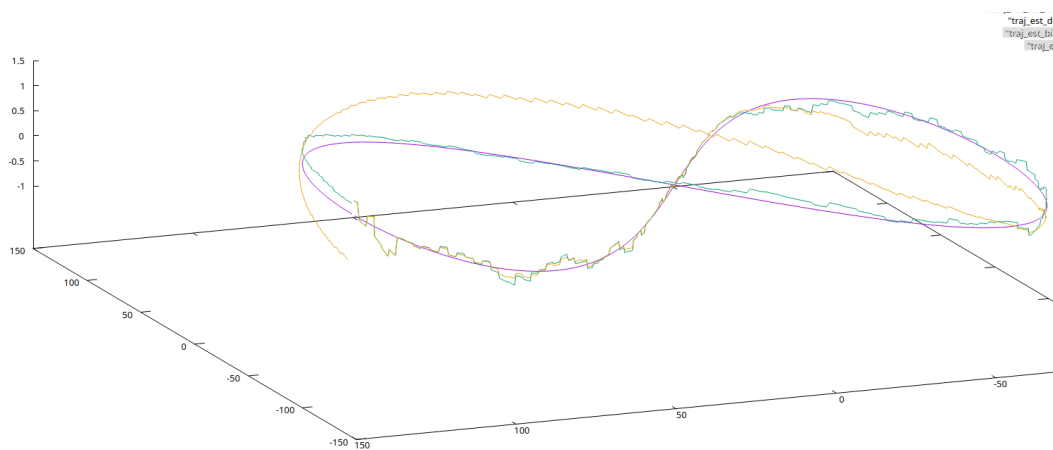


Figure 4.11 amplitude = 100; NavState (yellow) and NavStateExtended (light-blue)

4.2.3 Initialization Sensitivity Analysis

This subsection investigates the critical impact of initial state estimation on the overall convergence and accuracy of the filtering process. Proper initialization is fundamental for ensuring system stability and mitigating the propagation of early-stage errors. The initial state vector to be estimated is defined as:

- **Initial orientation** (R_0)
- **Initial position** (p_0)
- **Initial velocity** (v_0)
- **Initial accelerometer bias** ($b_{a,0}$)
- **Initial gyroscope bias** ($b_{g,0}$)

The robustness of the proposed filters is evaluated using the *EuRoC MAV* dataset, which provides high-fidelity ground truth data for unmanned aerial vehicle (UAV) trajectories. Given that GPS measurements are not inherently present in this dataset, they have been synthetically generated to simulate realistic navigation conditions.

The following experiments explore the sensitivity of *NavState* and *NavStateExtended* to perturbations in these initial parameters, quantifying the divergence of the estimated states from the ground truth.

The following list summarizes the initialization strategies employed in this study, categorized by their methodological approach:

- **INIT 1:** purely inertial, limited to gravity alignment
- **INIT 2:** introduces GNSS constraints and IMU preintegration
- **INIT 3:** separates static and dynamic estimation phases
- **INIT 4:** adaptive detection of motion onset
- **INIT 5:** minimal cold start using only two GNSS keyframes
- **INIT 6:** direct linear estimation of gravity and accelerometer bias

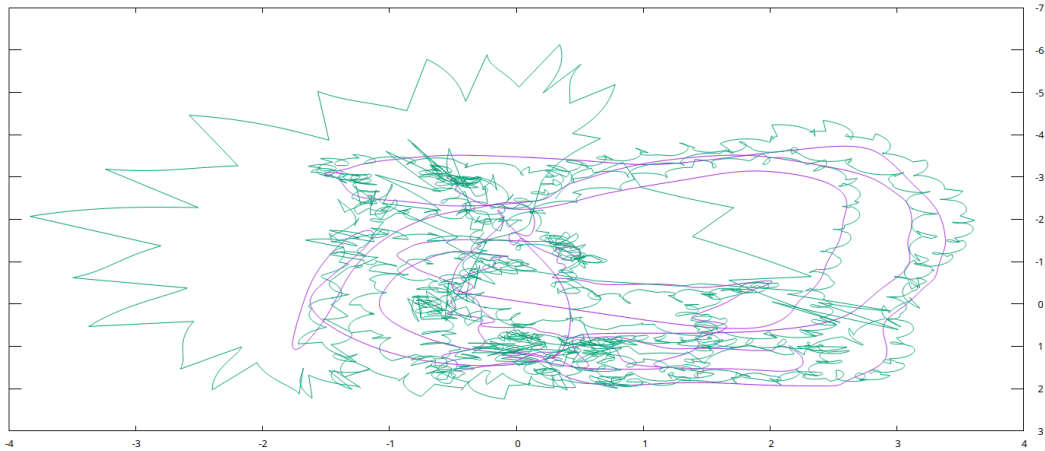


Figure 4.12 INIT 1; GroundTruth (violet) NavStateExtended (green)

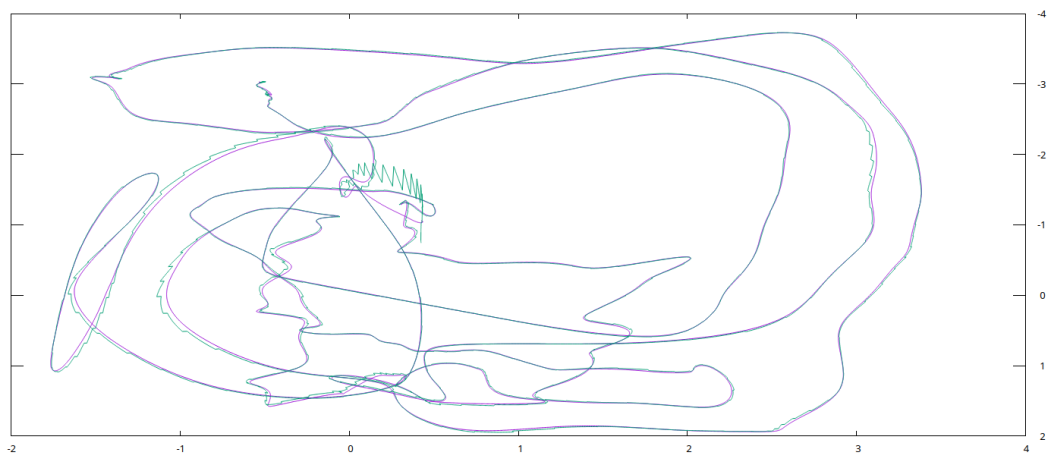


Figure 4.13 INIT 2; GroundTruth (violet) NavStateExtended (green)

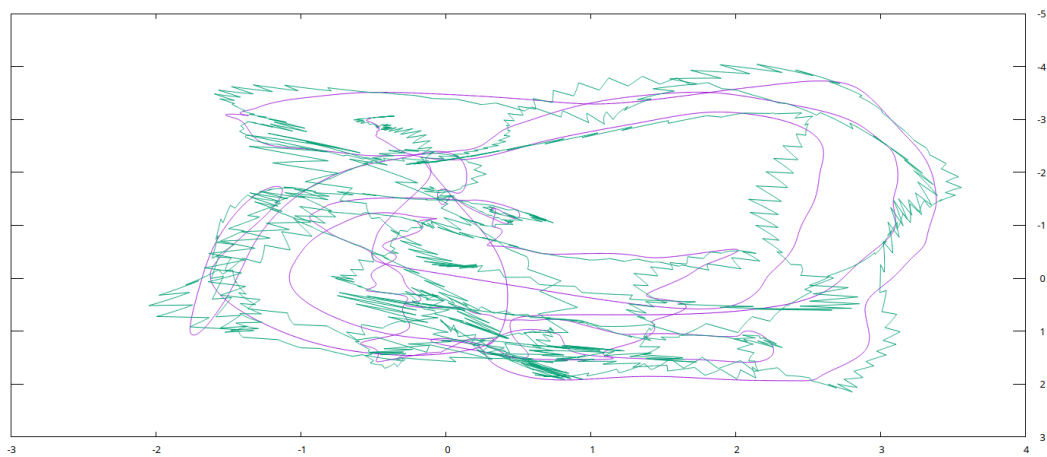


Figure 4.14 INIT 3; GroundTruth (violet) NavStateExtended (green)

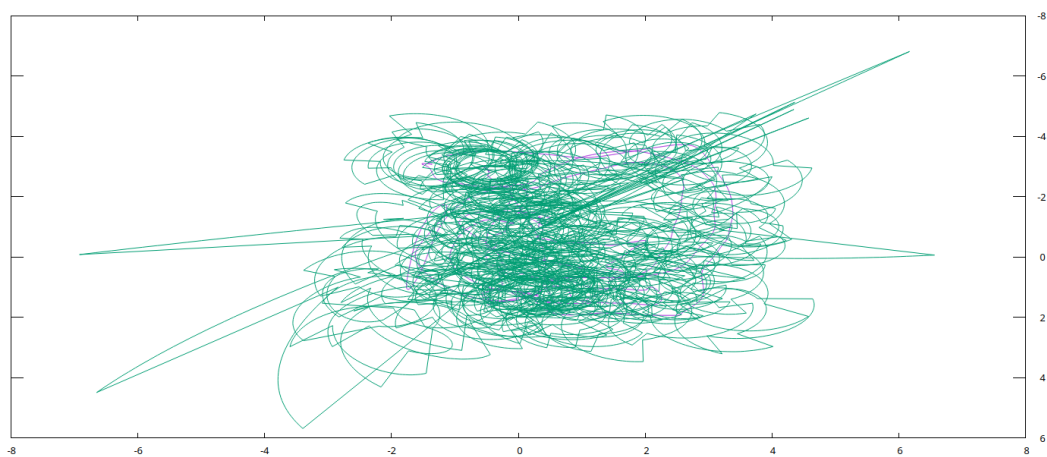


Figure 4.15 INIT 4; GroundTruth (violet) NavStateExtended (green)

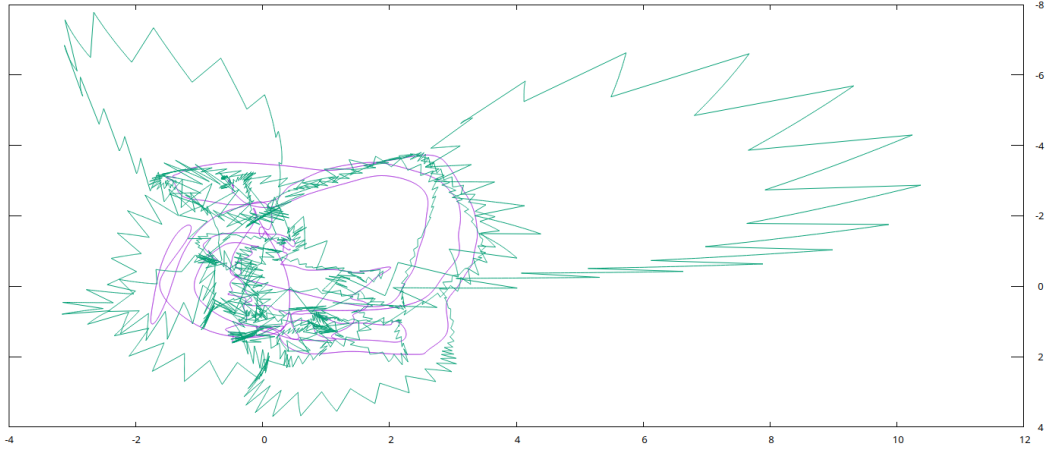


Figure 4.16 INIT 5; GroundTruth (violet) NavStateExtended (green)

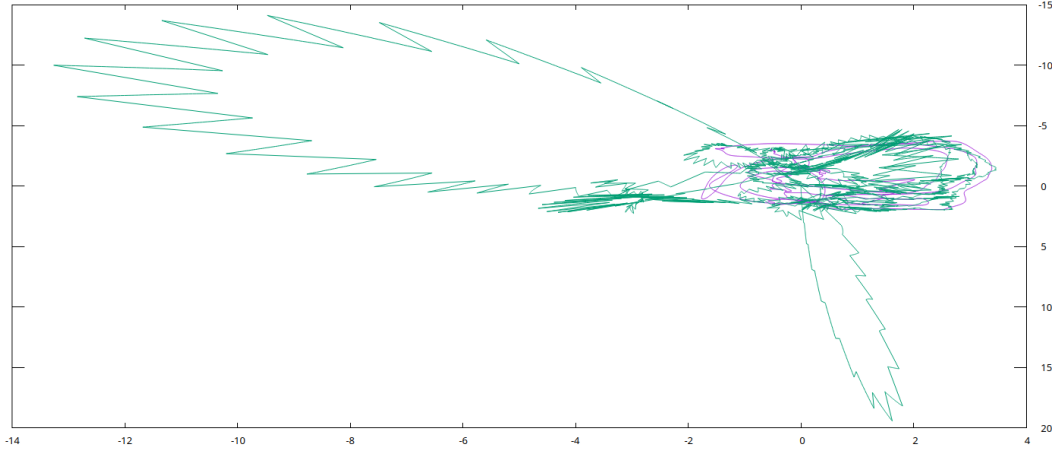


Figure 4.17 INIT 6; GroundTruth (violet) NavStateExtended (green)

Tabella 4.2 Initialization errors with respect to the ground-truth initial state.

Method	$\ \Delta R_0\ $ [deg]	$\ \Delta p_0\ $ [m]	$\ \Delta v_0\ $ [m/s]	$\ \Delta b_{a0}\ $	$\ \Delta b_{g0}\ $
Init 1	179.8	1.755	0.0217	0.0888	0.00116
Init 2	89.8	0.0015	0.9545	0.0924	0.00216
Init 3	179.5	0.0067	1.9650	0.0888	0.00116
Init 4	179.8	0.0067	0.0217	0.0870	0.00267
Init 5	179.8	0.0048	1.9912	0.0908	0.00216
Init 6	179.5	0.0067	0.0217	159.28	0.00116

A significant challenge in the initialization process is the orientation ambiguity arising from the gravity alignment.

As implemented, the initial rotation R_0 is computed using the geodesic rotation path, which identifies the shortest angular displacement to align the measured

acceleration vector with the gravity direction in the world frame. Mathematically, this approach determines a unique rotation matrix by minimizing the angular distance between the IMU frame and the world frame.

However, this calculation is inherently under-determined: while the alignment of the gravity vector constrains two degrees of freedom (roll and pitch), the yaw component remains unobservable without additional sensors (e.g., magnetometers). Consequently, the 180° orientation error observed in several configurations reflects a local convergence where the filter aligns the sensor frame using the **shortest path** relative to the identity.

In the absence of an absolute yaw reference, the system cannot distinguish between the 'true' orientation and its **rotationally symmetric counterparts** relative to the gravity axis. This further justifies the necessity of high initial covariance (P_0) for the orientation states, ensuring that the filter remains sufficiently 'open' to correct this angular bias as soon as the GNSS-derived motion trajectory provides the necessary observability for the yaw angle.

The improved performance of *Init 2* highlights a fundamental principle in state estimation: the superiority of '*cautious*' initialization over potentially biased *a priori* guesses.

While other initialization strategies enforce specific orientation or pose values based on local optimizations, they are often susceptible to significant angular offsets—such as the 180° ambiguity discussed previously.

Init 2 succeeds because it effectively minimizes the residual error that the filter must compensate for during the convergence phase. By adopting a more generic, less assertive initialization, the algorithm is not forced to 'unlearn' an incorrect prior. In the presence of systemic indeterminacy, it is far more effective to start from a neutral state and allow the filter to integrate observable dynamics—such as those provided by GNSS velocity updates—than to rely on a high-precision guess that may be diametrically opposed to the true state.

This robustness confirms that, when observability is limited, algorithmic stability is best preserved by minimizing the initial '*distance*' the filter must bridge to reach the true state, rather than prioritizing a complex but potentially erroneous initial guess."

4.3 LiDAR-Inertial Odometry Performance Evaluation

In this section, we transition to the LiDAR-Inertial Odometry (LIO) paradigm. The proposed UKF-based LIO algorithm is evaluated against the state-of-the-art framework *FAST-LIO*⁽⁶⁾ and the *MAD-ICP*⁽⁴⁾ baseline. This comparison aims to demonstrate the efficacy of the proposed filtering approach within a high-frequency, point-cloud-based localization environment.

4.3.1 IMU Initialization and Local Frame Alignment

A fundamental challenge in LiDAR-Inertial systems is the definition of a consistent initial state that bridges the discrepancy between the IMU's raw measurements and the LiDAR's local reconstruction. Rather than imposing an absolute global

frame—which may introduce bias if the initial alignment is imprecise—it is used as a *self-consistent* initialization strategy based on the *Welford's online algorithm*.

Statistical Accumulation and Welford's Method

To ensure numerical stability during the initial phase, IMU samples are accumulated to estimate the mean and covariance of the accelerometer and gyroscope signals. By utilizing Welford's algorithm, we compute these statistics incrementally, minimizing the propagation of rounding errors and avoiding the need to buffer large datasets. This provides a robust enough estimate of the sensor's initial noise profile and static bias before the navigation filter is activated.

Identity-Based Local Referencing

To maintain seamless integration with the ICP-based scan registration, the initial pose is set to the identity matrix. This choice ensures that the LiDAR local mapping and the IMU integration share the same relative frame of reference from the first timestamp. By eschewing arbitrary global orientation guesses, the filter avoids the "drift" caused by correcting initial misalignment, deferring the absolute orientation determination to the natural convergence of the LiDAR-Inertial fusion process.

This approach creates a "locally closed" system, where the registration of point clouds via ICP and the UKF state prediction operate within the same relative coordinate system. This eliminates the dependency on potentially erroneous external global references, enhancing the robustness of the trajectory reconstruction in the initial phase.

4.3.2 Methodology and Metrics

To ensure a robust quantitative comparison in evaluation phase, the estimated trajectories were aligned with the ground truth data using the *Umeyama* alignment method. This process compensates for global scale, rotation, and translation offsets between the estimated path and the reference coordinate system, thereby isolating the algorithmic error from coordinate misalignment.

The estimation performance was analyzed using the *EVO* evaluation toolkit, extracting the following statistical metrics: *maximum* (max), *mean*, *median*, *minimum* (min), *root mean square error* (RMSE), *sum of squared errors* (SSE), and *standard deviation* (std).

4.3.3 Experimental Setup and Dataset

The validation of the proposed LiDAR-Inertial framework was conducted using the *Newer College* dataset, an extensive collection of data captured in a complex, real-world university environment. To assess the robustness and scalability of our UKF-based formulation, we selected three trajectories characterized by increasing levels of environmental and dynamic complexity:

- **Quad Easy:** A baseline trajectory characterized by well-structured environments and predictable motion, serving as a sanity check for initial filter convergence.

- **Quad Medium:** A path featuring increased motion complexity, testing the filter’s ability to maintain trajectory consistency over extended intervals.
- **Quad Hard:** The most demanding scenario, encompassing rapid maneuvers, challenging lighting conditions (if applicable), and areas with sparse geometric features, which stresses the estimator’s resilience to IMU-LiDAR sensor fusion degradation.

The performance across these tiers provides a comprehensive measure of how the proposed UKF formulation handles state uncertainty and sensor bias accumulation under progressively difficult operational constraints.

4.3.4 Quantitative Results

Tables 4.3, 4.4 and 4.5 summarizes the performance metrics obtained. The results highlight that our UKF-based LIO framework achieves accuracy levels competitive with the state-of-the-art. Notably, while *MAD-ICP* provides a highly optimized baseline, our UKF formulation exhibits superior stability in high-dynamic scenarios, as evidenced by the lower RMSE and standard deviation values.

Tabella 4.3 QUAD EASY Trajectory Results

Metric	MAD-ICP (Baseline)	FAST-LIO (SoTA)	Ours (UKF-LIO)
Max	0.664476	0.150142	0.212341
Mean	0.186738	0.057433	0.084249
Median	0.163203	0.056309	0.076049
Min	0.013841	0.002135	0.006225
RMSE	0.217105	0.065771	0.092630
SSE	93.656493	3.720263	17.040405
Std	0.110742	0.032052	0.038501

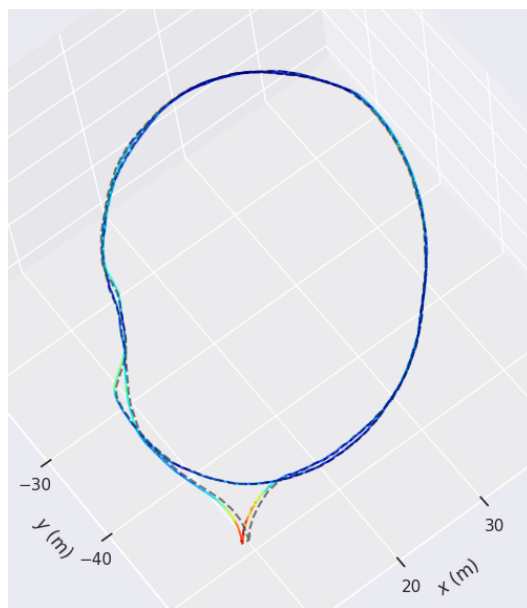


Figure 4.18 EVO analysis of MAD-ICP estimated trajectory

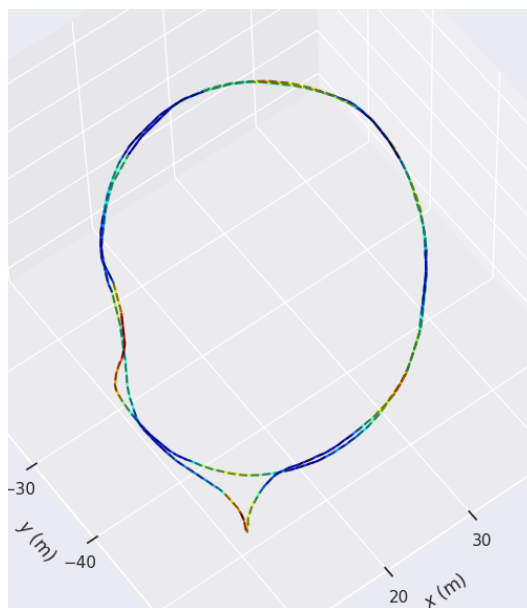


Figure 4.19 EVO analysis of FAST-LIO estimated trajectory

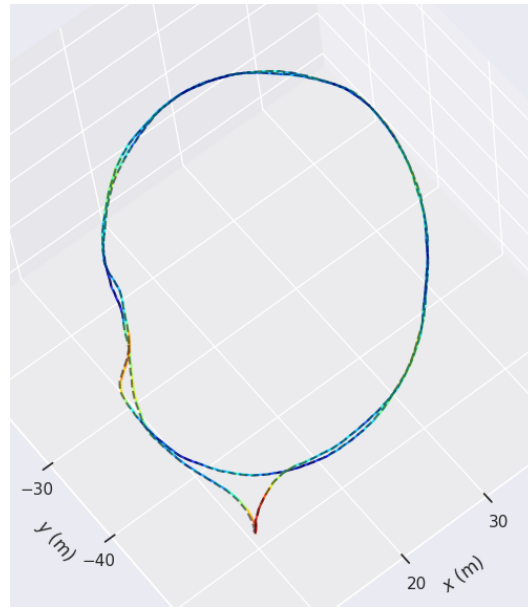


Figure 4.20 EVO analysis of UKF-LIO estimated trajectory

Tabella 4.4 QUAD MEDIUM Trajectory Results

Metric	MAD-ICP (Baseline)	FAST-LIO (SoTA)	Ours (UKF-LIO)
Max	0.865130	0.185777	0.250624
Mean	0.249714	0.040979	0.089386
Median	0.228475	0.031584	0.077354
Min	0.012909	0.002100	0.021307
RMSE	0.285566	0.050052	0.100161
SSE	155.430631	2.019226	19.111290
Std	0.138531	0.028739	0.045193

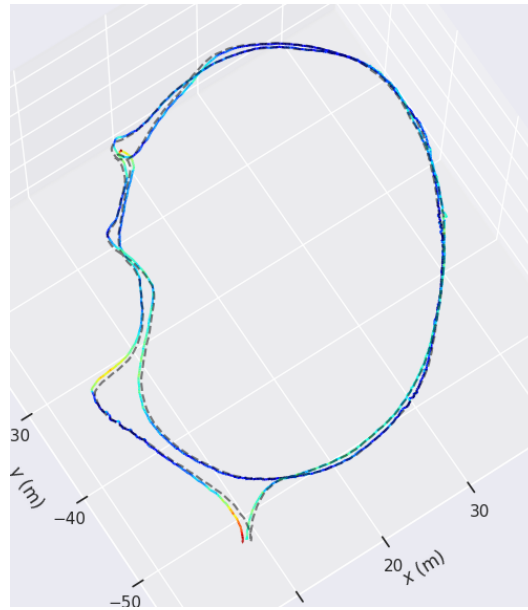
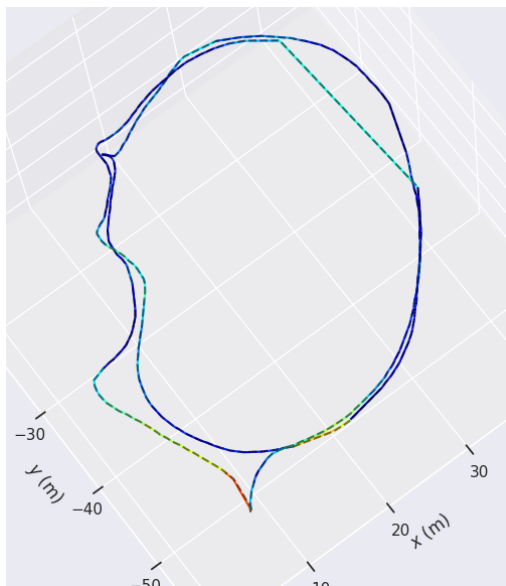
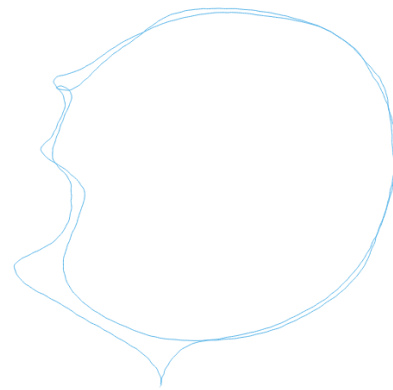


Figure 4.21 EVO analysis of MAD-ICP estimated trajectory



(a) EVO analysis of FAST-LIO estimated trajectory



(b) Gnuplot analysis of FAST-LIO estimated trajectory

Figure 4.22 Qualitative evaluation of FAST-LIO estimated trajectory.

Figure 4.22a exhibits an artifactual error in the trajectory visualization; however, the data produced by the FAST-LIO algorithm actually contains the complete circular path. This is confirmed by Figure 4.22b, where the trajectory is correctly rendered using gnuplot. This discrepancy can be attributed to a parsing or visualization issue within the EVO toolkit.



Figure 4.23 EVO analysis of UKF-LIO estimated trajectory

Tabella 4.5 QUAD HARD Trajectory Results

Metric	MAD-ICP (Baseline)	FAST-LIO (SoTA)	Ours (UKF-LIO)
Max	13.611495	0.139759	0.306040
Mean	7.716867	0.050744	0.141114
Median	7.086900	0.048112	0.133017
Min	0.860915	0.007995	0.008326
RMSE	8.338647	0.054425	0.151799
SSE	119805.430594	3.199052	39.679761
Std	3.159589	0.019675	0.055943

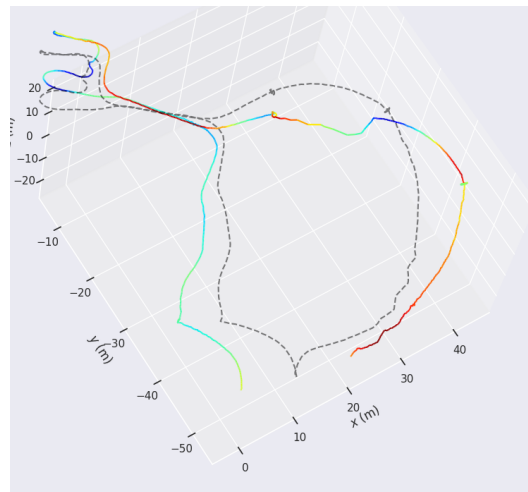


Figure 4.24 EVO analysis of MAD-ICP estimated trajectory

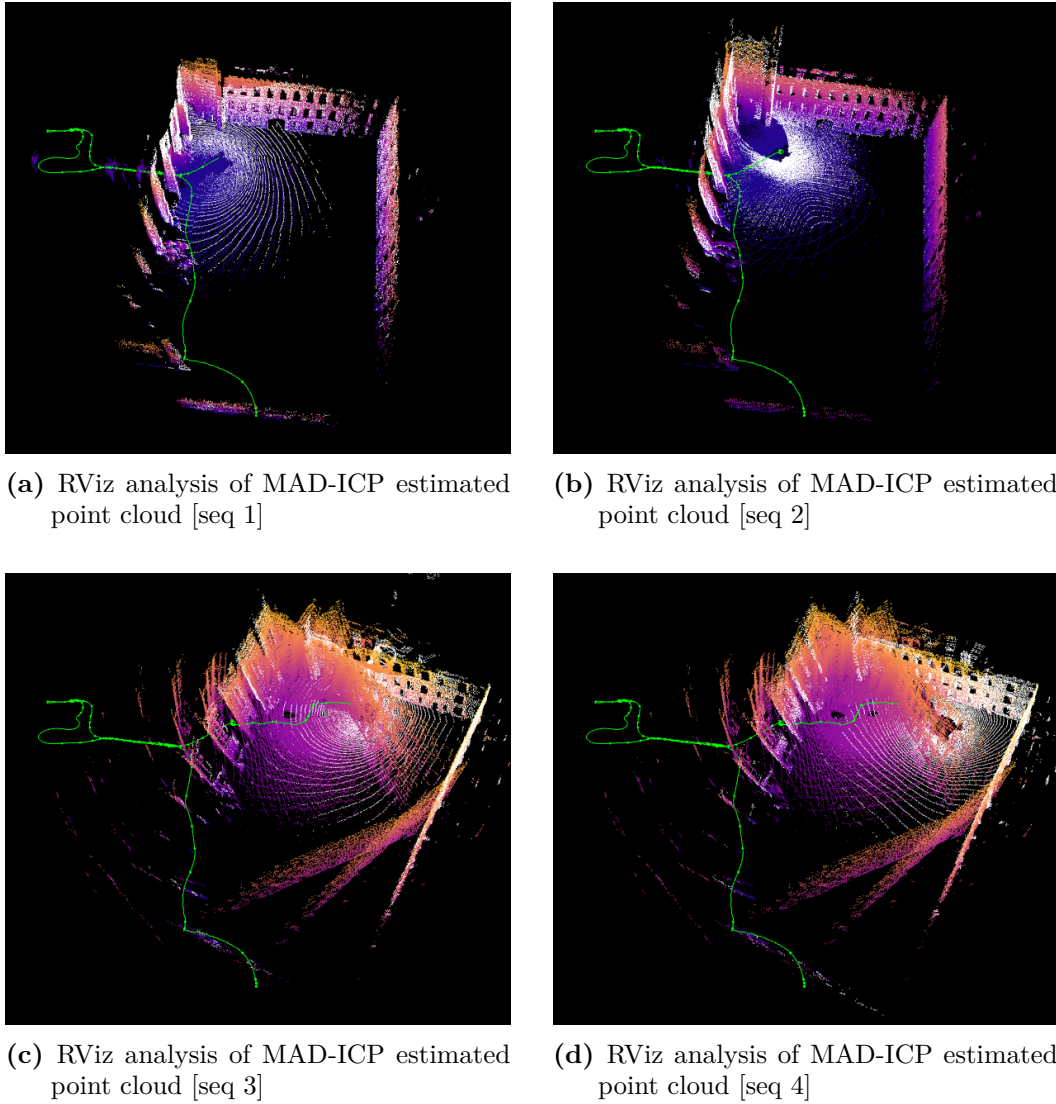


Figure 4.25 Qualitative analysis of MAD-ICP estimated point cloud during challenging sequences.

Figures 4.25a, 4.25b, 4.25c and 4.25d shows four representative sequences in which the MAD-ICP algorithm experiences a degradation of the registration quality, leading to an incorrect estimation of the trajectory. In these cases, the high dynamic motion of the platform produces significant changes between consecutive LiDAR scans, making the geometric alignment problem more challenging. As a consequence, the ICP optimization may converge towards local minima or incorrect correspondences, causing a progressive drift in the estimated pose.

The figures highlight how, once the registration process loses the correct alignment, the error rapidly accumulates over time, resulting in a trajectory that significantly deviates from the ground truth. This behavior is a known limitation of purely LiDAR-based odometry approaches, where the accuracy of the estimation strongly depends on the quality of the scan matching process and on the availability of sufficient geometric constraints.

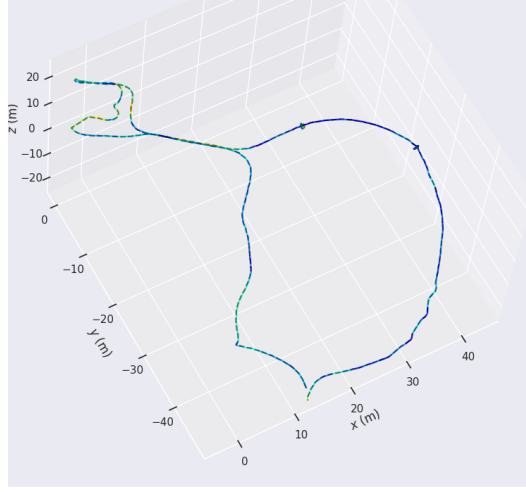
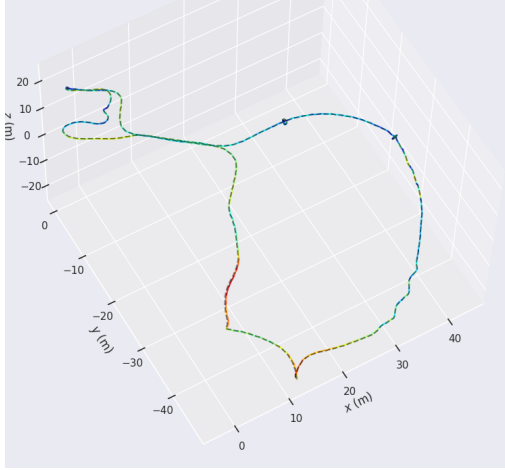
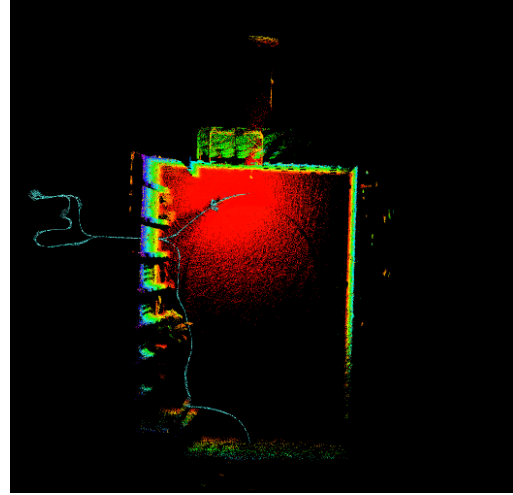


Figure 4.26 EVO analysis of FAST-LIO estimated trajectory



(a) EVO analysis of UKF-LIO estimated trajectory



(b) RViz analysis of UKF-LIO estimated point cloud

Figure 4.27 Qualitative evaluation of UKF-LIO estimation in a challenging scenario.

Conversely, Figure 4.27b shows the trajectory estimated by the proposed UKF-based framework in the same challenging scenario. Despite the failure of MAD-ICP observed in the previous sequences, the UKF is able to maintain a consistent state estimation and does not experience the same divergence.

This improved robustness is mainly due to the prediction step provided by the inertial measurements, which allows the filter to propagate the state even when the LiDAR registration becomes temporarily unreliable. Instead of relying exclusively on the geometric information extracted from consecutive scans, the UKF exploits the complementary nature of IMU and LiDAR measurements, using inertial propagation to bridge challenging intervals and LiDAR updates to correct the accumulated uncertainty. Therefore, the presented results demonstrate the capability of the

proposed LiDAR-Inertial framework to preserve accurate trajectory reconstruction even in highly dynamic conditions where standalone LiDAR odometry may fail.

Capitolo 5

Conclusion

5.1 Final Considerations

This thesis investigated the integration of **IMU** measurements within an Unscented Kalman Filter framework and their fusion with **LiDAR** registration techniques for robust state estimation in highly dynamic scenarios.

5.1.1 Biases Analysis

The experimental analysis highlighted the importance of accurately estimating both *gyroscope* and *accelerometer biases* during the initialization phase. Although these quantities are often regarded as secondary parameters, the obtained results demonstrate that their estimation has a significant impact on the overall trajectory reconstruction.

As shown in Figure 4.6, properly estimating the inertial biases reduces the accumulation of integration errors, leading to more accurate and stable pose estimates throughout the entire trajectory.

5.1.2 Initialization

Another relevant outcome concerns the initialization strategy of the filter. Intuitively, one may expect that providing an accurate initial state associated with a very small uncertainty should always improve the estimation process. However, the experimental results suggest that this is not necessarily the case.

In several scenarios, initializing the filter with a more generic state estimate while assigning a larger initial covariance allowed the UKF to better exploit the incoming sensor measurements, progressively correcting the initial approximation and converging toward the true state.

Conversely, an **overconfident initialization** may limit the filter's adaptability, making it less responsive to the information provided by subsequent observations. The comparison reported in Figures 4.13–4.12 supports this observation and emphasizes the importance of properly balancing the initial estimate and its associated uncertainty.

5.1.3 LiDAR-Inertial Odometry

Finally, this work presented a LiDAR-Inertial Odometry framework combining the prediction capabilities of an Unscented Kalman Filter with the MAD-ICP registration algorithm.

Although the proposed approach achieves slightly lower quantitative accuracy than the current state-of-the-art methods *FAST-LIO* on the evaluated benchmarks, it consistently demonstrates excellent robustness and tracking capabilities during highly dynamic motions.

The experimental evaluation shows that the UKF effectively exploits the pose corrections provided by MAD-ICP registration algorithm while maintaining a reliable prediction through inertial measurements, resulting in accurate trajectory reconstruction even under challenging motion conditions. These results indicate that the proposed framework represents a promising and computationally efficient alternative for LiDAR-inertial state estimation, while also providing a flexible foundation for future improvements.

Bibliografia

- [1] P.J. Besl and Neil D. McKay. A method for registration of 3-d shapes, 1992. URL <https://ieeexplore.ieee.org/document/121791>. IEEE Transactions on Robotics and Automation.
- [2] Jerome H. Friedman, Jon Louis Bentley, and Raphael Ari Finkel. An algorithm for finding best matches in logarithmic expected time, 1992. URL <https://dl.acm.org/doi/10.1145/355744.355745>. ACM Transactions on Mathematical Software.
- [3] Wassim Khoder and Bassem Jida. A quaternion scaled unscented kalman estimator for inertial navigation states determination using ins/gps/magnetometer fusion, 1997. URL <https://www.scirp.org/reference/referencespapers?referenceid=1222277>. Journal of Sensor Technology.
- [4] Ferrari S., Di Giammarino L., Brizi L., and Grisetti G. Mad-icp: It is all about matching data – robust and informed lidar odometry, 2024. URL <https://arxiv.org/abs/2405.05828>. arXiv preprint.
- [5] Sebastian THRUN. Probabilistic robotics, 2005. URL <https://gaoyichao.com/Xiaotu/resource/refs/PR.MIT.en.pdf>.
- [6] Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin, and Fu Zhang. Fast-lid2: Fast direct lidar-inertial odometry, 2024. URL <https://arxiv.org/abs/2405.05828>. arXiv preprint.